



江蘇大學
JIANGSU UNIVERSITY

数据结构课程设计报告

学院名称: 数学科学学院
专业班级: 数据计算及应用 2401
学生姓名: 李元博
学生学号: 3244301022
完成日期: 2026 年 1 月 8 日

2026 年 1 月

题目一：足球赛事票务系统

一、需求分析

1. 程序运行的环境：一般的 windows7 及以上系统，C98 环境（如 Devc++，至少需要包含 `<stdio.h>`、`<stdlib.h>`、`<time.h>`、`<string.h>`、`<math.h>` 等标准库）
2. 程序所实现的功能：订票，并且根据客户的存款身份、职位身份、先来后到的顺序进行预定座位，超出的座位在队列中等待，等待下一次分配到座位号。
3. 程序的输入：根据提示输入字符串 %s 或者整数 %d
4. 程序的输出：界面与提醒，以及打印出来的哈希表
5. 测试数据：形式如 "Alice", 1001, 0, 0, 0, 'n', 'y', 1000, NULL 的 Guest 的结构体，一共 20 个

二、设计说明

1. 算法设计思想：在未出票的时候，客户先订票，进入队列 Q1 中等待出票，出票时，先根据每个人的身份、订票时间进行排序，形成一个新的队列 Q2，将 Q2 中的元素依次出队，进入到哈希表中，哈希表用链地址法处理冲突。

2. 主要数据结构的说明：

（1）哈希表 HashTable

哈希表由两个部分构成，第一个是哈希表的元素 HashElement 结构体，该结构体有 3 个成员分别是 1. 哈希表的元素 2. 该节点的状态（即是否被占用）3. 指向哈希表的下一个节点的指针 另外一个哈希表的“表头”，该结构体也有 3 个成员，分别是 1. 空的表头，但是它的指针指向下一个哈希表的元素 2. 哈希表的空闲数量 3. 哈希表的节点总数量，不过这边还隐藏了一个量，就是已经被占用的哈希表节点数量。

此外，该哈希表的哈希函数为 $\text{HashFuntion}(x) = x \% (\text{HashFuntionContest} - 2)$ ，其中 HashFuntionContest 是先前定义的哈希表节点总数量。

（2）队列 Queue

使用 typedef 定义了两个抽象数据类型，分别是链表 struct QNode 与队列 struct Lin

其中，链表 struct QNode 定义如下：

该结构体包含两个结构体成员：

1. QElemType data 为链表（队列）的元素
2. struct QNode *next 为指向链表（队列）下一个元素的指针
3. QNode 为 struct QNode 的简称，而 *QueuePtr 为指向 struct QNode 的类型

```
typedef struct QNode{
```

```
    QElemType data;
```

```
    struct QNode *next;
```

```
}QNode,*QueuePtr;
```

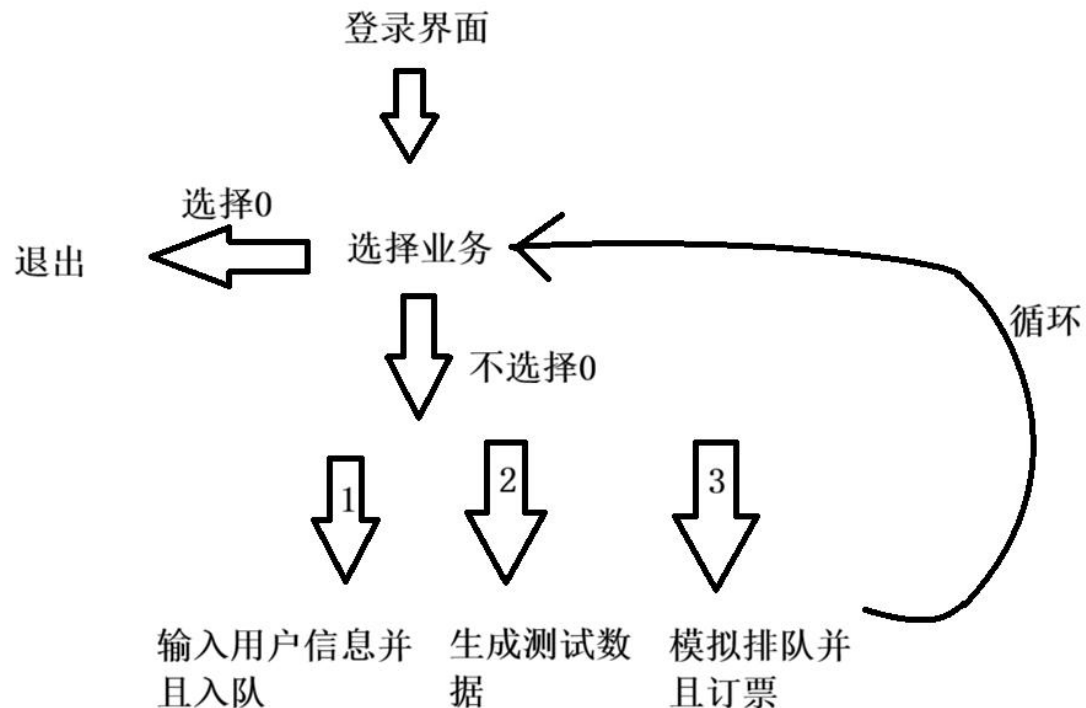
队列 struct Lin 定义如下：

该结构体包含两个结构体成员：

1. QueuePtr front 为指向队首的元素的指针
2. QueuePtr rear 为指向队尾的元素的指针

```
typedef struct Lin{
    QueuePtr front;
    QueuePtr rear;
}LinkQueue;
```

3. 程序主要流程图



主要模块说明：

登陆界面：这里需要用户选择需要操作的业务，用 `switch-case` 函数模块来实现，例如，选择 1 进入输入用户信息并且入队模块。

循环体：在用户未输入 0 的情况下，选择业务会一直循环进行直到用户选择退出系统

输入用户信息并且入队：引导观众输入自己的信息，进行判断，然后根据先后次序进行入队

生成测试数据：将测试数据导入进队列中。

模拟排队并且订票：将新的队列 Q2 进行依次入哈希表，直至队空或者表满

4. 程序的主要函数及其伪代码说明

```
int InitQueue(LinkQueue *Q)//初始化一个队列
Q->front=Q->rear=(QueuePtr)malloc(sizeof(QNode));
Int DeQueue(LinkQueue *Q,QElemType *e)//将队头元素 e 出队并且记录在 e 中返回
p=Q->front->next;*e=p->data;Q->front->next=p->next;
int EnQueue(LinkQueue *Q,QElemType e)//将元素 e 进行入队操作
p=(QueuePtr)malloc(sizeof(QNode));
p->data=e;p->next=NULL;Q->rear->next=p;Q->rear=p;
Void InsertHT(Hashtable *Ht, HashElemType g)//将元素 g 插入到哈希表中
HLink h = (HLink)malloc(sizeof(HElement));
HashElemType *newElem = (HashElemType*)malloc(sizeof(HashElemType));
newElem=g; h->elem = newElem;h->status = 1;h->hnext = NULL;
```

```

if(Ht->head[m] == NULL) {Ht->head[m] = h;}
else {HLink p = Ht->head[m];while(p->hnext != NULL) {p = p->hnext;}p->hnext = h;}
Ht->total++;

```

Void login(LinkQueue *Q)//登陆函数

```

printf("\n=== 观众登录 ===\n");
inputGuest(&g);
printf("\n 请检查你的个人信息是否完整:\n");
printGuest(&g);
printf("\n 请确认个人信息是否正确（正确输入 1，重新输入输入 0）:");
scanf("%d", &confirm);
if (confirm == 1) {
    if (EnQueue(Q, g)) {
        printf("\n 观众 %s 已成功加入队列\n", g.name);
        return 1;}
    else {
        printf("\n 入队失败\n");
        return 0;
    }
} else {
    printf("\n 已取消登录\n");
    return 0;
}

```

```

sortedBySeatnumber(LinkQueue *Q, Hashtable *ht)//按照规则排序
while(p) {InsertHT(ht, *p);p=p->next;}

```

三、源代码

```

#include<bits/stdc++.h>
#define ERROR 0
#define OK 1
#define value id
#define MAX_NAME_LEN 50
#define SEAT_AVAILABLE 0
#define SEAT_OCCUPIED 1
#define QElemType Guest
#define HashElemType Guest
#define HashFuntionContest 20
#define HTC 30

typedef struct Guest {
    char name[MAX_NAME_LEN];
    int id;

```

```

    int ticket;
    int score;
    int seat_number;
    char attribute;
    char importance;
    int savingmoney;
    struct Guest* next;
} Guest;

typedef struct HashElement {
    HashElemType *elem;
    int status;
    struct HashElement *hnext;
} HElement, *HLink;

typedef struct HashTable {
    HLink head[HashFuntionContest];
    int space;
    int total;
} Hashtable,*ht;

int HashFuntion(int num) {
    return num % (HashFuntionContest - 2);
}

typedef struct QNode{
    QElemType data;
    struct QNode *next;
}QNode,*QueuePtr;

typedef struct Lin{
    QueuePtr front;
    QueuePtr rear;
}LinkQueue;

int InitQueue(LinkQueue *Q){
    Q->front=Q->rear=(QueuePtr)malloc(sizeof(QNode));
    if(!Q->front) return 0;
    Q->front->next=NULL;
    return 1;
}

int EnQueue(LinkQueue *Q,QElemType e){
    QueuePtr p;

```

```

    p=(QueuePtr)malloc(sizeof(QNode));
    if(!p) return 0;
    p->data=e;
    p->next=NULL;
    Q->rear->next=p;
    Q->rear=p;
    return 1;
}

```

```

int EnRightQueueAttribute(LinkQueue *QA,LinkQueue *QE,LinkQueue *QN,Guest
g){
    switch (g.attribute) {
        case 'a':
            EnQueue(QA,g);
            break;
        case 'e':
            EnQueue(QE,g);
            break;
        case 'n':
            EnQueue(QN,g);
            break;
        default:
            printf("未知身份\n");
            return 0;
    }
    return 1;
}

```

```

int EnRightQueueImportance(LinkQueue *QS,LinkQueue *QV,LinkQueue *QY,Guest
g){
    switch (g.importance) {
        case 'y':
            EnQueue(QY,g);
            break;
        case 'v':
            EnQueue(QV,g);
            break;
        case 's':
            EnQueue(QS,g);
            break;
        default:
            printf("未知身份\n");
            return 0;
    }
}

```

```

        return 1;
    }

int DeQueue(LinkQueue *Q,QElemType *e){
    QueuePtr p;
    if(Q->front==Q->rear)return 0;
    p=Q->front->next;
    *e=p->data;
    Q->front->next=p->next;
    if(Q->rear==p) Q->rear=Q->front;
    free(p);
    return 1;
}

int GetHead(LinkQueue *Q,QElemType *e){
    if(Q->front==Q->rear)return 0;
    *e=Q->front->next->data;
    return 1;
}

int QueueEmpty(LinkQueue *Q){
    return Q->front==Q->rear;
}

int QueueLength(LinkQueue *Q){
    int length=0;
    QueuePtr p=Q->front->next;
    while(p){
        length++;
        p=p->next;
    }
    return length;
}

int ClearQueue(LinkQueue *Q){
    QueuePtr p,q;
    p=Q->front->next;
    while(p){
        q=p->next;
        free(p);
        p=q;
    }
    Q->front->next=NULL;
    Q->rear=Q->front;
}

```

```

        return 1;
    }

int DestroyQueue(LinkQueue *Q){
    while(Q->front){
        Q->rear=Q->front->next;
        free(Q->front);
        Q->front=Q->rear;
    }
    return 1;
}

int myprint(LinkQueue *Q){
    QueuePtr q=Q->front->next;
    if(!q) printf("the queue is empty");
    while(q){
        printf("(id:%d,name:%s,seat:%d)  ", q->data.id, q->data.name,
q->data.seat_number);
        q=q->next;
    }
    printf("\n");
    return 1;
}

void initHashTable(Hashtable **Ht) {
    *Ht = (Hashtable*)malloc(sizeof(Hashtable));
    int i = 0;
    for(i = 0; i < HashFuntionContest; i++) {
        (*Ht)->head[i] = NULL;
    }
    (*Ht)->total = 0;
    (*Ht)->space = HTC;
}

void InsertHT(Hashtable *Ht, HashElemType g) {
    int m = HashFuntion(g.id);
    HLink h = (HLink)malloc(sizeof(HElement));
    HashElemType *newElem = (HashElemType*)malloc(sizeof(HashElemType));

    strcpy(newElem->name, g.name);
    newElem->ticket = g.ticket;
    newElem->id = g.id;
    newElem->seat_number = g.seat_number;
    newElem->attribute = g.attribute;

```



```

newElem->score = g.score;
newElem->importance = g.importance;
newElem->savingmoney = g.savingmoney;
newElem->next = g.next;

h->elem = newElem;
h->status = 1;
h->hnext = NULL;

if(Ht->head[m] == NULL) {
    Ht->head[m] = h;
} else {
    HLink p = Ht->head[m];
    while(p->hnext != NULL) {
        p = p->hnext;
    }
    p->hnext = h;
}
Ht->total++;
}

void PrintHashTable(Hashtable *Ht) {
    int i = 0;
    for(i = 0; i < HashFuntionContest; i++) {
        printf("Bucket[%d]: ", i);
        HLink p = Ht->head[i];
        while(p != NULL) {
            if(p->elem != NULL) {
                printf("(id:%d,name:%s) ", p->elem->id, p->elem->name);
            }
            p = p->hnext;
        }
        printf("\n");
    }
    printf("Total elements: %d\n", Ht->total);
}

void FreeHashTable(Hashtable *Ht) {
    int i = 0;
    for(i = 0; i < HashFuntionContest; i++) {
        HLink p = Ht->head[i];
        while(p != NULL) {
            HLink temp = p;
            p = p->hnext;
        }
    }
}

```

```

        if(temp->elem != NULL) {
            free(temp->elem);
        }
        free(temp);
    }
}
free(Ht);
}

void generate_attribute(Guest *g) {
    int p = rand() % 100 + 1;
    if (p == 1) {
        g->attribute = 'a';
    } else if (p <= 11) {
        g->attribute = 'e';
    } else {
        g->attribute = 'n';
    }
}

void print_attribute(const Guest *g) {
    switch (g->attribute) {
        case 'a':
            printf("职业身份是: 军人\n");
            break;
        case 'e':
            printf("职业身份是: 教师\n");
            break;
        case 'n':
            printf("职业身份是: 群众\n");
            break;
        default:
            printf("未知职业身份\n");
    }
}

void determine_vip(Guest *g) {
    if (g->savingmoney >= 4000) {
        g->importance = 's';
    } else if (g->savingmoney >= 2000) {
        g->importance = 'v';
    } else {
        g->importance = 'y';
    }
}

```

```
}
```

```
void print_vip(const Guest *g) {  
    switch (g->importance) {  
        case 's':  
            printf("身份是：超级会员\n");  
            break;  
        case 'v':  
            printf("身份是：会员\n");  
            break;  
        case 'y':  
            printf("身份是：普通观众\n");  
            break;  
        default:  
            printf("未知会员等级\n");  
    }  
}
```

```
void inputGuest(Guest *g) {  
    printf("请输入你的姓名：");  
    getchar();  
    scanf("%[^\\n]", g->name);  
    printf("请输入你的证件号：");  
    scanf("%d", &(g->id));  
    printf("请输入你的储蓄金额：");  
    scanf("%d", &(g->savingmoney));  
    //getchar();  
    generate_attribute(g);  
    determine_vip(g);  
    g->ticket = 0;  
    g->seat_number = 0;  
    g->score = 0;  
    g->next = NULL;  
}
```

```
void printGuest(Guest *g) {  
    printf("====身份检查====\n");  
    printf("姓名： %s\n", g->name);  
    printf("证件号： %d\n", g->id);  
    printf("你的");  
    print_vip(g);  
    printf("你的");  
    print_attribute(g);  
    printf("预约状态： %s\n", g->ticket ? "已预约" : "未预约");  
}
```

```

    if (g->ticket) {
        printf("座位号: %d\n", g->seat_number);
    }
    printf("=====\n");
}

int login(LinkQueue *Q) {
    Guest g;
    int confirm;

    printf("\n=== 观众登录 ===\n");
    inputGuest(&g);
    printf("\n 请检查你的个人信息是否完整:\n");
    printGuest(&g);

    printf("\n 请确认个人信息是否正确（正确输入 1，重新输入输入 0）: ");
    scanf("%d", &confirm);
    //getchar();

    if (confirm == 1) {
        if (EnQueue(Q, g)) {
            printf("\n 观众 %s 已成功加入队列\n", g.name);
            return 1;
        } else {
            printf("\n 入队失败\n");
            return 0;
        }
    } else {
        printf("\n 已取消登录\n");
        return 0;
    }
}

Guest* setSeatNumber(LinkQueue *Q){
    int score=1;
    int flag=1;
    Guest* p = NULL;
    Guest* L = NULL;
    Guest g;

    while(!QueueEmpty(Q)){
        DeQueue(Q, &g);

        if(flag){

```

```

        L = (Guest*)malloc(sizeof(Guest));
        *L = g;
        L->next = NULL;
        flag = 0;
        p = L;
    } else {
        Guest* newNode = (Guest*)malloc(sizeof(Guest));
        *newNode = g;
        newNode->next = NULL;
        p->next = newNode;
        p = newNode;
    }

    p->seat_number = score + p->importance + p->attribute;
    score++;
}
return L;
}

```

```

void sortBySeatNumber(LinkQueue *Q){
    Guest* Head = (Guest*)malloc(sizeof(Guest));
    Head->next = setSeatNumber(Q);

    Guest* current = Head->next;
    while(current != NULL){
        Guest* min = current;
        Guest* preMin = Head;
        Guest* p = current;
        Guest* pre = Head;

        while(p != NULL){
            if(p->seat_number < min->seat_number){
                min = p;
                preMin = pre;
            }
            pre = p;
            p = p->next;
        }

        if(min != current){
            preMin->next = min->next;
            min->next = current;
            Head->next = min;
        }
    }
}

```

```

        current = min->next;
    }

    Guest* sorted = Head->next;
    while(sorted != NULL){
        EnQueue(Q, *sorted);
        Guest* temp = sorted;
        sorted = sorted->next;
        free(temp);
    }
    free(Head);
}

void sortedBySeatnumber(LinkQueue *Q, Hashtable *ht){
    Guest* L=setSeatNumber(Q);
    Guest* p=L;
    while(p){
        InsertHT(ht, *p);
        p=p->next;
    }
}

void TestFunData(Hashtable *Ht, LinkQueue *Q){
    Guest elem1 = {"Alice", 1001, 0, 0, 0, 'n', 'y', 1000, NULL};
    Guest elem2 = {"Bob", 1002, 0, 0, 0, 'e', 'v', 3000, NULL};
    Guest elem3 = {"Charlie", 1003, 0, 0, 0, 'n', 's', 5000, NULL};
    Guest elem4 = {"David", 1010, 0, 0, 0, 'a', 'y', 1500, NULL};
    Guest elem5 = {"Eve", 1011, 0, 0, 0, 'n', 'v', 2500, NULL};
    Guest elem6 = {"Frank", 1005, 0, 0, 0, 'e', 'y', 1000, NULL};
    Guest elem7 = {"Alice2", 1001, 0, 0, 0, 'n', 'y', 1000, NULL};
    Guest elem8 = {"David2", 1010, 0, 0, 0, 'a', 'y', 1500, NULL};
    Guest elem9 = {"Michael", 1012, 0, 0, 0, 'e', 's', 2000, NULL};
    Guest elem10 = {"Sophia", 1013, 0, 0, 0, 'a', 'v', 4500, NULL};
    Guest elem11 = {"Liam", 1014, 0, 0, 0, 'n', 'y', 800, NULL};
    Guest elem12 = {"Emma", 1015, 0, 0, 0, 'e', 's', 6000, NULL};
    Guest elem13 = {"Noah", 1016, 0, 0, 0, 'a', 'y', 1800, NULL};
    Guest elem14 = {"Olivia", 1017, 0, 0, 0, 'n', 'v', 2200, NULL};
    Guest elem15 = {"Lucas", 1018, 0, 0, 0, 'e', 'y', 1200, NULL};
    Guest elem16 = {"Ava", 1019, 0, 0, 0, 'a', 's', 4200, NULL};
    Guest elem17 = {"Elijah", 1020, 0, 0, 0, 'n', 'y', 900, NULL};
    Guest elem18 = {"Mia", 1021, 0, 0, 0, 'e', 'v', 3800, NULL};
    Guest elem19 = {"Ethan", 1022, 0, 0, 0, 'a', 'y', 1600, NULL};
    Guest elem20 = {"Charlotte", 1023, 0, 0, 0, 'n', 's', 4800, NULL};
}

```

```

    EnQueue(Q, elem1);
    EnQueue(Q, elem2);
    EnQueue(Q, elem3);
    EnQueue(Q, elem4);
    EnQueue(Q, elem5);
    EnQueue(Q, elem6);
    EnQueue(Q, elem7);
    EnQueue(Q, elem8);
    EnQueue(Q, elem9);
    EnQueue(Q, elem10);
    EnQueue(Q, elem11);
    EnQueue(Q, elem12);
    EnQueue(Q, elem13);
    EnQueue(Q, elem14);
    EnQueue(Q, elem15);
    EnQueue(Q, elem16);
    EnQueue(Q, elem17);
    EnQueue(Q, elem18);
    EnQueue(Q, elem19);
    EnQueue(Q, elem20);

    printf("插入测试数据完成! \n");
}

int main() {
    srand((unsigned int)time(NULL));
    Hashtable *ht = NULL;
    LinkQueue Q;
    int conti=1;
    int flag =1;
    printf("Initialized hash table and queue.\n");
    InitQueue(&Q);initHashTable(&ht);
    //printf("Running comprehensive test...\n");
    printf("\n");
    while(conti){
        printf("===请输入需要办理的业务: ===\n");
        printf("1.登陆并且订票(第二个人在登陆时姓名前需要加上空格)\n");
        printf("2.生成测试数据\n");
        printf("3.开始模拟排队订票+进入\n");
        printf("0.退出系统\n");
        printf("请输入编号: ");
        scanf("%d",&flag);
        printf("\n");
    }
}

```

```

switch(flag){
    case 1:{login(&Q);break;}
    case 2:TestFunData(ht, &Q);break;
    case 3:sortedBySeatnumber(&Q,ht);printf(" 座位 哈 希 表 如 图 所 示 :
\n");PrintHashTable(ht);break;
    case 0:conti=0;
}
}

printf("\nCleaning up...\n");
//FreeHashTable(ht);
ClearQueue(&Q);

printf("Test completed.\n");

return 0;
}

```

四、上机结果及体会

1. 实际完成情况说明：

完成了简单的出票以及公平排序等功能，只要有合理的设计，就可以支持以后的数据扩展

2. 程序的性能分析，包括时空分析

哈希表插入操作：时间复杂度为 $O(n)$ ，空间复杂度为 $O(n + m)$

队列插入删除操作：时间复杂度为 $O(1)$ ，空间复杂度为 $O(n)$

sortBySeatNumber 函数时间复杂度为 $O(n^2)$

3. 打印程序运行时的初值和运行结果，画出相应的图示

```

===请输入需要办理的业务：===
1.登陆并且订票(第二个人在登陆时姓名前需要加上空格)
2.生成测试数据
3.开始模拟排队订票+进入
0.退出系统
请输入编号：2

插入测试数据完成！

```


插入测试数据完成！

===请输入需要办理的业务：===

1. 登陆并且订票(第二个人在登陆时姓名前需要加上空格)

2. 生成测试数据

3. 开始模拟排队订票+进入

0. 退出系统

请输入编号：3

座位哈希表如图所示：

Bucket[0]:

Bucket[1]:

Bucket[2]: (id:1010,name:David) (id:1010,name:David2)

Bucket[3]: (id:1011,name:Eve)

Bucket[4]: (id:1012,name:Michael)

Bucket[5]: (id:1013,name:Sophia)

Bucket[6]: (id:1014,name:Liam)

Bucket[7]: (id:1015,name:Emma)

Bucket[8]: (id:1016,name:Noah)

Bucket[9]: (id:1017,name:Olivia)

Bucket[10]: (id:1018,name:Lucas)

Bucket[11]: (id:1001,name:Alice) (id:1001,name:Alice2) (id:1019,name:Ava)

Bucket[12]: (id:1002,name:Bob) (id:1020,name:Elijah)

Bucket[13]: (id:1003,name:Charlie) (id:1021,name:Mia)

Bucket[14]: (id:3200,name:zhu fe) (id:1022,name:Ethan)

Bucket[15]: (id:1005,name:Frank) (id:1023,name:Charlotte)

Bucket[16]:

Bucket[17]:

Bucket[18]:

Bucket[19]:

Total elements: 21

===请输入需要办理的业务：===

1. 登陆并且订票(第二个人在登陆时姓名前需要加上空格)

2. 生成测试数据

3. 开始模拟排队订票+进入

0. 退出系统

请输入编号：

```

1.登陆并且订票(第二个人在登陆时姓名前需要加上空格)
2.生成测试数据
3.开始模拟排队订票+进入
0.退出系统
请输入编号：1

=== 观众登录 ===
请输入你的姓名：zhu fe
请输入你的证件号：3200
请输入你的储蓄金额：50000

请检查你的个人信息是否完整：
=====身份检查=====
姓名：zhu fe
证件号：3200
你的身份是：超级会员
你的职业身份是：教师
预约状态：未预约
=====

请确认个人信息是否正确（正确输入1，重新输入输入0）：1

```

4. 上机过程中出现的问题及其解决方案

问题 1:

参考文献中给的哈希表无法直接匹配到我的系统中来

可以引进新的函数变量 `gust g` 来适配原来的函数，以及将用链表实现的哈希表改造成为用顺序存储的结构，并且用二次探测处理冲突，在不改变函数结构的基础上达到函数的改造

问题 2: 发现系统里面出现大量 `if` 与 `if else` 判断语句

解决方案: 换用 `case` 来替代，使得语言更加简洁明了

问题 3: 输入输出不正确

解决方案: 使用 `%[^\\n]` 格式符替代 `'scanf'` 读取带空格的字符串。

5. 程序中可以改进的地方说明

排序算法: 时间复杂度太高，可以考虑采用归并排序

哈希表优化: 改进哈希函数减少冲突（如采用 `djb2` 算法），增加动态扩容机制，使用链地址法并优化冲突处理逻辑，修复初始化不完整导致的空指针问题；

内存管理: 修复 `EnQueue` 失败时的内存泄漏，添加 `FreeHashTable` 释放哈希表内存，优化 `Guest` 结构体内存池预分配；

队列操作: 增加队列容量检查防止溢出，修复 `EnQueue` 未更新队尾指针的隐患，优化多属性分组逻辑；

测试数据: 改用动态数据生成替代硬编码测试用例，增加随机性测试覆盖；

循环控制: 修复主循环 `conti` 变量未修改导致的死循环，添加 `default` 分支处理非法输入；

代码结构: 封装重复代码为函数（如哈希表操作、排序算法），减少全局变量依赖，采用模块化设计；

错误处理: 完善内存分配检查，添加输入验证机制，增强哈希表负载因子监控

6. 程序中可以扩充的功能及设计实现假想

可以尝试改进加入删除哈希表元素操作，这样程序会更加符合实际情况，而且补位的时候会更加明显一些。

增加一些查找类型的函数，如根据 id 查找观众的座位号，根据座位号查找观众的姓名等等这样的函数，可以方便管理。

7. 收获及体会

这次座位预约系统的课程设计，让我深刻体会到：独立完成一套功能完整的系统开发远非易事。开发过程中不仅会陷入思路瓶颈，新增功能时还常与既有逻辑产生冲突——每一个功能的拓展都需要兼顾原有流程的兼容性，这让我意识到系统设计“全局统筹”的重要性。这是我首次独立负责规模较大的系统开发，过程中不得不兼顾数据规模带来的时间复杂度问题，更要考虑计算机内存的有限性而优化空间占用。

此外，解决 BUG 的过程也让我收获颇丰：例如在处理用户姓名的格式化输入时，我遇到了%s格式符遇到空格就终止读取的问题，导致无法完整录入带空格的姓名。通过查阅 CSDN 博客的技术教程，我最终用%[\n]格式符解决了这一问题，这也让我切实体会到“主动查阅资料”是开发者必备的能力。值得一提的是，我还借助 AI 工具辅助调试代码错误，大幅提升了问题解决的效率；而哈希表的核心实现逻辑，也是通过参考 CSDN 博客的技术分享才得以理清思路——这些工具与资源，都是开发过程中不可或缺的助力

五、参考文献

C 语言实现的火车票预订系统开发指南 CSDN

https://blog.csdn.net/weixin_42594427/article/details/142629336

C/C++中带空格的字符串的输入 CSDN

<https://blog.csdn.net/circle2015/article/details/70880072>

指针空间的申请和释放_释放指针所指内存空间

https://blog.csdn.net/2401_87557001/article/details/143728880

C++中 Txt 文件读取和写入(方法汇总)

<https://blog.csdn.net/lz20120808/article/details/49622787>

解决哈希冲突（四种方法）

https://blog.csdn.net/qq_48241564/article/details/118613312

中国人民解放军现役军人主要数据

中国人民解放军现役军人的民族构成（人）

<https://www.stats.gov.cn/sj/pcsj/rkpc/5rp/html/append12.htm>

中国教育在线.2024 年全国共有专任教师近 1900 万人

题目二： 镇江旅游导览系统

一、需求分析

1、 运行环境（软、硬件环境）：

一般的 windows7 及以上系统，C++98 环境（如 Devc++，至少需要包含 <stdio.h>、<stdlib.h>、<limits.h>、<string.h> 、<math.h>等标准库）

2、 程序所实现的功能：创建固定图的邻接表表示、生成同类型景点的子图、查找景点并且计算从江苏大学到指定景点的最短路径。

3、 程序的输入：根据提示输入字符串%s 或者整数%d。

4、 程序的输出： 图的邻接矩阵表示，路径的规划，景点介绍

5、 测试数据：形如“城上村遗址”，1，'h'，NULL 的邻接表节点

二、设计说明

1、 算法设计的思想：

首先，查阅给定的 26 个景点之间的距离，发现两两景点之间距离很大，于是我给定了一个阈值，20km，如果超过 20km，那么我们就认为这两个景点之间是不相邻的，因此，这里的无穷大我们也用 20km 来表示。这样一来我们就得到了一幅普通的图，一共有 37 条边，每个边的权重就是两个景点之间的距离（保留一位小数）。

然后，发现这张图不连通，根据查阅的资料，将两条离 20km 最近割边加入，并且令这两条新加入的边为 20km，让其勉强变为一个连通图，后面我们用迪杰斯特拉算法计算会方便许多。

2、 主要的数据结构设计说明：

由于这个图对应的邻接矩阵是一个稀疏矩阵，会占用较大空间，所以我采用邻接表来完成对这个图的构建，以及对后续需要用到的景点类型进行一系列的定義。

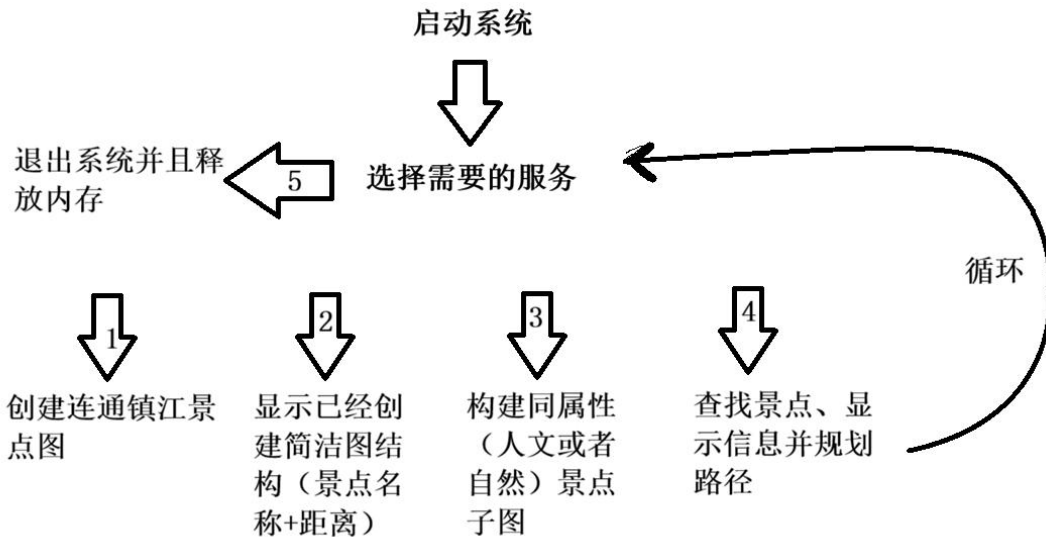
下面有一个图 ALGraph（用邻接表）的定义：这个图有三个成员，分别是：

1. 邻接表的表头(数据类型为 VNode，即邻接表节点) 2. 顶点的数量 (int) 3. 边的数量 (int)

对于邻接表（节点）的定义，有四个成员，分别是：

1. 景点的名称 2. 景点的编号（方便查找） 3. 景点的类型（人文，自然，或者两者都有）
4. 与景点相邻的节点及其边的权重（用指针表示）

3、 程序的主要流程图：



4、程序的主要模块

创建连通镇江景点图:初始化一个图,为内置的镇江景点图,并且将其加上合理的边变为连通图
显示简洁图结构:仅仅显示每个节点与其邻接点的关系与距离

构建同属性景点子图:将同一属性的节点筛选出来,用二者的最短路径作为两者之间的距离,做成子图,然后根据子图从一个顶点出发,根据迪杰斯特拉算法,给出遍历子图的最短路径,完成参观相同属性景点的旅行。

查找景点、显示信息并规划路径:给出从江苏大学到给定顶点的最短路径,并且给出给定顶点的景点介绍。

5、程序的主要函数及其伪代码说明 (不需要完整的代码)

//求最短路径: 迪杰斯特拉算法

```

void Dijkstra(const ALGraph* G, int start, double* dist, int* prev)
{
    n = G.vexnum
    Visited[i] = -1
    dist[i] = INFINITY;
    For(i=0; i<n; i++) {u = min{distance}; If(!u) break; Visited[u]=1;}
    while(IsEdge(u, v))
    {
        if( visited[v]==0 && dist[u] + weight < dist[v])
        {
            dist[v] = dist[u] + weight; prev[v] = u;
        }
        Free(visited)
    }
}
  
```

//深度遍历图 DFS

```

void DFS(const ALGraph* G, int v, int* visited, int* component, int* compSize)
{
    Visited[v]=1;
    Component+=v;
    compSize++;
    u=v->next;
    If(!visited[u])
    DFS(G, adj, visited, component, compSize);
}
  
```

三、源代码

1. 主函数及所有子函数(完整的代码)

```
#include<bits/stdc++.h>
#define MVNum 100
#define OK 1
#define ERROR 0
#define INF 1000000.0

typedef int Status;
typedef char VerTexType[50];

typedef struct ArcNode {
    int adjvex;
    double weight;
    struct ArcNode* nextarc;
} ArcNode;

typedef struct VNode {
    VerTexType data;
    int id;
    char type;
    ArcNode* firstarc;
} VNode, AdjList[MVNum];

typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;

typedef struct EdgeInfo {
    char v1[50];
    char v2[50];
    double weight;
} EdgeInfo;

typedef struct ScenicIntro {
    char name[50];
    char description[500];
} ScenicIntro;

int LocateVexALG(const ALGraph* G, const char* v) {
    int i;
    for ( i = 0; i < G->vexnum; ++i) {
```

```

        if (strcmp(G->vertices[i].data, v) == 0) {
            return i;
        }
    }
    return -1;
}

int IsEdgeExists(const ALGraph* G, int i, int j) {
    ArcNode* p = G->vertices[i].firstarc;
    while (p != NULL) {
        if (p->adjvex == j) {
            return 1;
        }
        p = p->nextarc;
    }
    return 0;
}

void AddEdge(ALGraph* G, int i, int j, double weight, int* edgeCount) {
    if (i == -1 || j == -1) {
        printf("错误: 无效的顶点索引\n");
        return;
    }

    if (i == j) {
        printf("警告: 尝试添加自环边, 已跳过\n");
        return;
    }

    if (IsEdgeExists(G, i, j)) {
        return;
    }

    ArcNode* p1 = (ArcNode*)malloc(sizeof(ArcNode));
    p1->adjvex = j;
    p1->weight = weight;
    p1->nextarc = G->vertices[i].firstarc;
    G->vertices[i].firstarc = p1;

    ArcNode* p2 = (ArcNode*)malloc(sizeof(ArcNode));
    p2->adjvex = i;
    p2->weight = weight;
    p2->nextarc = G->vertices[j].firstarc;
    G->vertices[j].firstarc = p2;
}

```

```

        (*edgeCount)++;
    }

Status CreateUDG(ALGraph* G) {
    int i,k;
    printf("\n 请输入顶点个数: ");
    scanf("%d", &G->vexnum);
    printf("请输入边的个数: ");
    scanf("%d", &G->arcnum);

    printf("请依次输入每个顶点的 *名称* *编号* *属性(h/e/b)*, 以空格分隔: \n");
    for ( i = 0; i < G->vexnum; ++i) {
        scanf("%s %d %c", G->vertices[i].data, &G->vertices[i].id,
&G->vertices[i].type);
        G->vertices[i].firstarc = NULL;
    }

    printf("请输入%d 条边所连接的两个顶点的名称和权重, 以空格分隔: \n", G->arcnum);
    int edgeCount = 0;
    for (k = 0; k < G->arcnum; ++k) {
        char v1[50], v2[50];
        double weight;
        scanf("%s %s %lf", v1, v2, &weight);
        int i = LocateVexALG(G, v1);
        int j = LocateVexALG(G, v2);

        if (i == -1 || j == -1) {
            printf("输入了不存在的顶点, 请检查. \n");
            continue;
        }

        AddEdge(G, i, j, weight, &edgeCount);
    }

    G->arcnum = edgeCount / 2;
    return OK;
}

void PrintGraph(const ALGraph* G) {
    int i;
    printf("\n===== 图结构信息 (邻接表) =====\n");
    printf("顶点数量: %d, 边数量: %d\n", G->vexnum, G->arcnum);
    printf("-----\n");
}

```



```

for ( i = 0; i < G->vexnum; ++i) {
    printf("名称: \"%s\\\"", G->vertices[i].data);
    printf(" -> 邻接点[名称,权重]: ");

    ArcNode* p = G->vertices[i].firstarc;
    int degree = 0;
    int first = 1;
    while (p != NULL) {
        int adjIndex = p->adjvex;
        if (!first) printf(", ");
        printf("%s,%.11fkm", G->vertices[adjIndex].data, p->weight);
        p = p->nextarc;
        degree++;
        first = 0;
    }
    if (degree == 0) {
        printf("无邻接点");
    }
    printf("\\n");
}
printf("=====\\n\\n");
}

void PrintGraphSimple(const ALGraph* G) {
    int i;
    printf("\\n===== 简洁图结构信息 (邻接表) =====\\n");
    printf("顶点数量: %d, 边数量: %d\\n", G->vexnum, G->arcnum);
    printf("-----\\n");
    for (i = 0; i < G->vexnum; ++i) {
        printf("%s -> 邻接点: ", G->vertices[i].data);

        ArcNode* p = G->vertices[i].firstarc;
        int first = 1;
        while (p != NULL) {
            int adjIndex = p->adjvex;
            if (!first) printf(", ");
            printf("%s(%.11fkm)", G->vertices[adjIndex].data, p->weight);
            p = p->nextarc;
            first = 0;
        }
        printf("\\n");
    }
    printf("=====\\n\\n");
}

```

```

void PrintByType(const ALGraph* G, char type) {
    int i;
    const char* typeName;
    switch(type) {
        case 'h': typeName = "人文类景点"; break;
        case 'e': typeName = "自然类景点"; break;
        case 'b': typeName = "双属性景点"; break;
        default: typeName = "未知类型景点"; break;
    }

    printf("\n=== %s (共%d 个景点) ===\n", typeName, G->vexnum);
    int count = 0;
    for (i = 0; i < G->vexnum; ++i) {
        if (G->vertices[i].type == type) {
            printf("%d. %s (编号:%d, 索引:%d)\n", ++count, G->vertices[i].data,
G->vertices[i].id, i);
        }
    }
    if (count == 0) {
        printf("未找到%s\n", typeName);
    } else {
        printf("总计: %d 个%s\n", count, typeName);
    }
}

void DestroyGraph(ALGraph* G) {
    int i,k;
    for ( i = 0; i < G->vexnum; ++i) {
        ArcNode* p = G->vertices[i].firstarc;
        while (p != NULL) {
            ArcNode* temp = p;
            p = p->nextarc;
            free(temp);
        }
        G->vertices[i].firstarc = NULL;
    }
    G->vexnum = 0;
    G->arcnum = 0;
    printf("图内存已释放\n");
}

```

```

void DFS(const ALGraph* G, int v, int* visited, int* component, int* compSize)
{
    visited[v] = 1;
    component[(*compSize)++] = v;

    ArcNode* p = G->vertices[v].firstarc;
    while (p != NULL) {
        int adj = p->adjvex;
        if (!visited[adj]) {
            DFS(G, adj, visited, component, compSize);
        }
        p = p->nextarc;
    }
}

```

```

int** GetConnectedComponents(const ALGraph* G, int* componentCount, int*
compSizes) {
    int i;
    int* visited = (int*)calloc(G->vexnum, sizeof(int));
    int** components = (int**)malloc(sizeof(int*) * G->vexnum);

    *componentCount = 0;

    for (i = 0; i < G->vexnum; ++i) {
        if (!visited[i]) {
            compSizes[*componentCount] = 0;
            components[*componentCount] = (int*)malloc(sizeof(int) *
G->vexnum);
            DFS(G, i, visited, components[*componentCount],
&compSizes[*componentCount]);
            (*componentCount)++;
        }
    }

    free(visited);
    return components;
}

```

```

void CreateZhenjiangWeightedGraph(ALGraph* G) {
    int i,k;
    printf("\n=== 创建带权重的镇江景点图===\n");

    G->vexnum = 26;
    G->arcnum = 0;
}

```

```

VNode vertices[] = {
    {"城上村遗址", 1, 'h', NULL},
    {"镇江醋文化博物馆", 2, 'h', NULL},
    {"永安陵石刻", 3, 'h', NULL},
    {"修安陵石刻", 4, 'h', NULL},
    {"访仙镇", 5, 'h', NULL},
    {"吕城镇", 6, 'h', NULL},
    {"金山寺", 7, 'h', NULL},
    {"北固楼", 8, 'h', NULL},
    {"西津渡古街", 9, 'h', NULL},
    {"焦山定慧寺", 10, 'h', NULL},
    {"甘露寺铁塔", 11, 'h', NULL},
    {"镇江天主教堂", 12, 'h', NULL},
    {"铁瓮城遗址", 13, 'h', NULL},
    {"镇江体育会展中心", 14, 'h', NULL},
    {"镇江站南广场", 15, 'h', NULL},
    {"句容余坤开元大酒店", 16, 'h', NULL},
    {"苏宁广场", 17, 'h', NULL},
    {"江苏大学", 18, 'h', NULL},
    {"镇江火车站", 19, 'h', NULL},
    {"镇江市政府", 20, 'h', NULL},
    {"江苏茶博园", 21, 'e', NULL},
    {"南山森林公园", 22, 'b', NULL},
    {"宝华山森林公园", 23, 'b', NULL},
    {"葛仙观", 24, 'b', NULL},
    {"茅山景区", 25, 'b', NULL},
    {"敕赐九霄万福宫", 26, 'b', NULL}
};

for ( i = 0; i < G->vexnum; ++i) {
    G->vertices[i] = vertices[i];
}

const int edgeCount = 37;
EdgeInfo edgeArray[37];

strcpy(edgeArray[0].v1, "苏宁广场"); strcpy(edgeArray[0].v2, "镇江市政府");
edgeArray[0].weight = 1.2;
strcpy(edgeArray[1].v1, "镇江市政府"); strcpy(edgeArray[1].v2, "镇江站南广场");
edgeArray[1].weight = 5.8;
strcpy(edgeArray[2].v1, "镇江站南广场"); strcpy(edgeArray[2].v2, "镇江体育会展中心");
edgeArray[2].weight = 2.3;

```

```
strcpy(edgeArray[3].v1, "镇江体育会展中心"); strcpy(edgeArray[3].v2, "南山  
森林公园"); edgeArray[3].weight = 3.1;  
strcpy(edgeArray[4].v1, "苏宁广场"); strcpy(edgeArray[4].v2, "镇江站南广场  
"); edgeArray[4].weight = 4.5;  
strcpy(edgeArray[5].v1, "镇江市政府"); strcpy(edgeArray[5].v2, "江苏大学");  
edgeArray[5].weight = 6.7;  
strcpy(edgeArray[6].v1, "镇江站南广场"); strcpy(edgeArray[6].v2, "江苏大学  
"); edgeArray[6].weight = 8.9;  
  
strcpy(edgeArray[7].v1, "金山寺"); strcpy(edgeArray[7].v2, "西津渡古街");  
edgeArray[7].weight = 0.8;  
strcpy(edgeArray[8].v1, "西津渡古街"); strcpy(edgeArray[8].v2, "北固楼");  
edgeArray[8].weight = 2.1;  
strcpy(edgeArray[9].v1, "北固楼"); strcpy(edgeArray[9].v2, "焦山定慧寺");  
edgeArray[9].weight = 3.6;  
strcpy(edgeArray[10].v1, "西津渡古街"); strcpy(edgeArray[10].v2, "甘露寺铁  
塔"); edgeArray[10].weight = 0.6;  
strcpy(edgeArray[11].v1, "西津渡古街"); strcpy(edgeArray[11].v2, "镇江天主  
教堂"); edgeArray[11].weight = 0.5;  
strcpy(edgeArray[12].v1, "北固楼"); strcpy(edgeArray[12].v2, "铁瓮城遗址");  
edgeArray[12].weight = 1.9;  
strcpy(edgeArray[13].v1, "金山寺"); strcpy(edgeArray[13].v2, "铁瓮城遗址");  
edgeArray[13].weight = 3.2;  
  
strcpy(edgeArray[14].v1, "南山森林公园"); strcpy(edgeArray[14].v2, "镇江醋  
文化博物馆"); edgeArray[14].weight = 7.4;  
strcpy(edgeArray[15].v1, "南山森林公园"); strcpy(edgeArray[15].v2, "西津渡  
古街"); edgeArray[15].weight = 5.3;  
strcpy(edgeArray[16].v1, "镇江醋文化博物馆"); strcpy(edgeArray[16].v2, "镇  
江站南广场"); edgeArray[16].weight = 4.9;  
  
strcpy(edgeArray[17].v1, "江苏大学"); strcpy(edgeArray[17].v2, "北固楼");  
edgeArray[17].weight = 3.8;  
strcpy(edgeArray[18].v1, "江苏大学"); strcpy(edgeArray[18].v2, "焦山定慧寺  
"); edgeArray[18].weight = 4.2;  
strcpy(edgeArray[19].v1, "镇江站南广场"); strcpy(edgeArray[19].v2, "金山寺  
"); edgeArray[19].weight = 3.4;  
  
strcpy(edgeArray[20].v1, "茅山景区"); strcpy(edgeArray[20].v2, "敕赐九霄万  
福宫"); edgeArray[20].weight = 2.5;  
strcpy(edgeArray[21].v1, "敕赐九霄万福宫"); strcpy(edgeArray[21].v2, "葛仙  
观"); edgeArray[21].weight = 3.7;  
strcpy(edgeArray[22].v1, "茅山景区"); strcpy(edgeArray[22].v2, "葛仙观");  
edgeArray[22].weight = 4.1;
```

```

        strcpy(edgeArray[23].v1, "江苏茶博园"); strcpy(edgeArray[23].v2, "茅山景区");
        edgeArray[23].weight = 8.2;
        strcpy(edgeArray[24].v1, "句容余坤开元大酒店"); strcpy(edgeArray[24].v2, "江苏茶博园");
        edgeArray[24].weight = 9.7;
        strcpy(edgeArray[25].v1, "句容余坤开元大酒店"); strcpy(edgeArray[25].v2, "茅山景区");
        edgeArray[25].weight = 17.9;

        strcpy(edgeArray[26].v1, "城上村遗址"); strcpy(edgeArray[26].v2, "永安陵石刻");
        edgeArray[26].weight = 4.8;
        strcpy(edgeArray[27].v1, "永安陵石刻"); strcpy(edgeArray[27].v2, "修安陵石刻");
        edgeArray[27].weight = 1.1;
        strcpy(edgeArray[28].v1, "修安陵石刻"); strcpy(edgeArray[28].v2, "访仙镇");
        edgeArray[28].weight = 6.3;

        strcpy(edgeArray[29].v1, "访仙镇"); strcpy(edgeArray[29].v2, "吕城镇");
        edgeArray[29].weight = 6.0;
        strcpy(edgeArray[30].v1, "城上村遗址"); strcpy(edgeArray[30].v2, "访仙镇");
        edgeArray[30].weight = 10.9;

        strcpy(edgeArray[31].v1, "南山森林公园"); strcpy(edgeArray[31].v2, "宝华山森林公园");
        edgeArray[31].weight = 15.0;
        strcpy(edgeArray[32].v1, "宝华山森林公园"); strcpy(edgeArray[32].v2, "茅山景区");
        edgeArray[32].weight = 12.0;
        strcpy(edgeArray[33].v1, "江苏茶博园"); strcpy(edgeArray[33].v2, "宝华山森林公园");
        edgeArray[33].weight = 8.5;
        strcpy(edgeArray[34].v1, "西津渡古街"); strcpy(edgeArray[34].v2, "镇江醋文化博物馆");
        edgeArray[34].weight = 5.5;
        strcpy(edgeArray[35].v1, "江苏大学"); strcpy(edgeArray[35].v2, "镇江火车站");
        edgeArray[35].weight = 3.0;
        strcpy(edgeArray[36].v1, "镇江火车站"); strcpy(edgeArray[36].v2, "镇江站南广场");
        edgeArray[36].weight = 1.5;

        int addedEdgeCount = 0;
        for ( k = 0; k < edgeCount; ++k) {
            int i = LocateVexALG(G, edgeArray[k].v1);
            int j = LocateVexALG(G, edgeArray[k].v2);
            double weight = edgeArray[k].weight;

            if (i != -1 && j != -1) {
                AddEdge(G, i, j, weight, &addedEdgeCount);
            } else {
                printf("警告: 边连接 %s - %s 中包含无效顶点, 已跳过此边\n",
                    edgeArray[k].v1, edgeArray[k].v2);
            }
        }
    }
}

```

```

    }
}

G->arcnum = addedEdgeCount / 2;

printf("带权重的镇江景点图创建完成! \n");
printf("包含 %d 个顶点和 %d 条边\n", G->vexnum, G->arcnum);
}

void CreateConnectedZhenjiangGraph(ALGraph* G) {
    int i;
    CreateZhenjiangWeightedGraph(G);

    int componentCount = 0;
    int* compSizes = (int*)malloc(sizeof(int) * G->vexnum);
    int** components = GetConnectedComponents(G, &componentCount, compSizes);

    if (componentCount > 1) {
        printf("\n 检测到 %d 个连通分量, 正在添加连接边...\n", componentCount);

        int edgeCount = G->arcnum * 2;

        int zhenjiangIdx = LocateVexALG(G, "苏宁广场");
        int jurongIdx = LocateVexALG(G, "茅山景区");
        int danyangIdx = LocateVexALG(G, "城上村遗址");

        if (zhenjiangIdx != -1 && jurongIdx != -1 && danyangIdx != -1) {
            printf("添加连接边: 苏宁广场 <-> 茅山景区 (权重: 20.0km)\n");
            AddEdge(G, zhenjiangIdx, jurongIdx, 20.0, &edgeCount);

            printf("添加连接边: 茅山景区 <-> 城上村遗址 (权重: 20.0km)\n");
            AddEdge(G, jurongIdx, danyangIdx, 20.0, &edgeCount);

            G->arcnum = edgeCount / 2;

            printf("\n 连通图创建完成! \n");
            printf("添加了 2 条连接边, 使整个图连通.\n");
        } else {
            printf("错误: 找不到区域代表景点\n");
        }
    } else {
        printf("图已经是连通图, 无需添加连接边.\n");
    }
}

```

```

    for (i = 0; i < componentCount; ++i) {
        free(components[i]);
    }
    free(components);
    free(compSizes);

    componentCount = 0;
    compSizes = (int*)malloc(sizeof(int) * G->vexnum);
    components = GetConnectedComponents(G, &componentCount, compSizes);

    printf("最终连通分量数量: %d\n", componentCount);
    if (componentCount == 1) {
        printf("√ 图是连通图\n");
    } else {
        printf("× 图不是连通图\n");
    }
}

for (i = 0; i < componentCount; ++i) {
    free(components[i]);
}
free(components);
free(compSizes);
}

void Dijkstra(const ALGraph* G, int start, double* dist, int* prev) {
    int i, j;
    int n = G->vexnum;
    int* visited = (int*)calloc(n, sizeof(int));

    for (i = 0; i < n; ++i) {
        dist[i] = INF;
        prev[i] = -1;
    }
    dist[start] = 0;

    for (i = 0; i < n; ++i) {
        int u = -1;
        double minDist = INF;
        for (j = 0; j < n; ++j) {
            if (!visited[j] && dist[j] < minDist) {
                minDist = dist[j];
                u = j;
            }
        }
    }
}

```



```

        if (u == -1) break;

        visited[u] = 1;

        ArcNode* p = G->vertices[u].firstarc;
        while (p != NULL) {
            int v = p->adjvex;
            double weight = p->weight;

            if (!visited[v] && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                prev[v] = u;
            }
            p = p->nextarc;
        }
    }

    free(visited);
}

void BuildSameAttributeSubgraph(ALGraph* G) {
    int i,g,j,count,v;
    printf("\n=== 构建同属性景点子图 ===\n");

    if (G->vexnum == 0) {
        printf("请先创建图! \n");
        return;
    }

    int* groups[3];
    int groupSizes[3] = {0, 0, 0};

    for ( i = 0; i < 3; ++i) {
        groups[i] = (int*)malloc(sizeof(int) * G->vexnum);
    }

    for (i = 0; i < G->vexnum; ++i) {
        if (G->vertices[i].type == 'h') {
            groups[0][groupSizes[0]++] = i;
        } else if (G->vertices[i].type == 'e') {
            groups[1][groupSizes[1]++] = i;
        } else if (G->vertices[i].type == 'b') {
            groups[2][groupSizes[2]++] = i;
        }
    }
}

```

```

    }
}

for ( g = 0; g < 3; ++g) {
    if (groupSizes[g] < 2) {
        continue;
    }

    const char* typeName;
    if (g == 0) typeName = "人文类景点";
    else if (g == 1) typeName = "自然类景点";
    else typeName = "双属性景点";

    printf("\n%s 子图 (包含%d 个景点):\n", typeName, groupSizes[g]);

    int groupSize = groupSizes[g];
    double** distanceMatrix = (double**)malloc(sizeof(double*) *
groupSize);
    for ( i = 0; i < groupSize; ++i) {
        distanceMatrix[i] = (double*)malloc(sizeof(double) * groupSize);
        for ( j = 0; j < groupSize; ++j) {
            distanceMatrix[i][j] = INF;
        }
    }

    for ( i = 0; i < groupSize; ++i) {
        int vertex_i = groups[g][i];
        double* dist = (double*)malloc(sizeof(double) * G->vexnum);
        int* prev = (int*)malloc(sizeof(int) * G->vexnum);
        Dijkstra(G, vertex_i, dist, prev);

        for ( j = 0; j < groupSize; ++j) {
            int vertex_j = groups[g][j];
            distanceMatrix[i][j] = dist[vertex_j];
        }

        free(dist);
        free(prev);
    }

    printf("景点列表: ");
    for ( i = 0; i < groupSize; ++i) {
        printf("%s", G->vertices[groups[g][i]].data);
        if (i < groupSize - 1) printf(", ");
    }
}

```

```

}
printf("\n");

printf("最短路径距离矩阵:\n");
printf("%20s", " ");
for ( j = 0; j < groupSize; ++j) {
    printf("%10.8s", G->vertices[groups[g][j]].data);
}
printf("\n");

for ( i = 0; i < groupSize; ++i) {
    printf("%20s", G->vertices[groups[g][i]].data);
    for ( j = 0; j < groupSize; ++j) {
        if (distanceMatrix[i][j] >= INF) {
            printf("%10s", "INF");
        } else {
            printf("%10.11f", distanceMatrix[i][j]);
        }
    }
    printf("\n");
}

printf("\n 建议的同属性景点游览路线 (基于最短路径连接):\n");
int* inTree = (int*)calloc(groupSize, sizeof(int));
int* parent = (int*)malloc(sizeof(int) * groupSize);
double* minDist = (double*)malloc(sizeof(double) * groupSize);

for ( i = 0; i < groupSize; ++i) {
    parent[i] = -1;
    minDist[i] = INF;
}
minDist[0] = 0;

for ( count = 0; count < groupSize; ++count) {
    int u = -1;
    double minVal = INF;
    for ( i = 0; i < groupSize; ++i) {
        if (!inTree[i] && minDist[i] < minVal) {
            minVal = minDist[i];
            u = i;
        }
    }

    if (u == -1) break;

```

```

        inTree[u] = 1;

        for ( v = 0; v < groupSize; ++v) {
            if (!inTree[v] && distanceMatrix[u][v] < minDist[v]) {
                minDist[v] = distanceMatrix[u][v];
                parent[v] = u;
            }
        }
    }

    printf("连接边 (起点 -> 终点 [距离]):\n");
    for ( i = 1; i < groupSize; ++i) {
        if (parent[i] != -1) {
            printf("  %s -> %s [%.11fkm]\n",
                G->vertices[groups[g][parent[i]]].data,
                G->vertices[groups[g][i]].data,
                distanceMatrix[parent[i]][i]);
        }
    }
    printf("\n");

    for ( i = 0; i < groupSize; ++i) {
        free(distanceMatrix[i]);
    }
    free(distanceMatrix);
    free(inTree);
    free(parent);
    free(minDist);
}

for ( i = 0; i < 3; ++i) {
    free(groups[i]);
}
}

const char* GetScenicIntroduction(const char* spotName) {
    int i,j,k,v,g;

    static struct {
        char name[50];
        char description[500];
    } introMap[] = {
        {"江苏大学", "学校坚持以贡献谋发展, 不断提升科技创新与服务社会能力。"},

```

{ "金山寺", "金山寺始建于东晋，是中国佛教禅宗名寺，民间传说《白蛇传》中的金山寺即指此。"},

{ "焦山定慧寺", "焦山定慧寺始建于东汉兴平年间，是江南佛教圣地之一。寺内古树参天，殿宇巍峨，碑刻林立。"},

{ "北固楼", "北固楼位于镇江市北固山上，与岳阳楼、黄鹤楼并称长江三大名楼。"},

{ "西津渡古街", "西津渡古街是镇江文物古迹保存最多、最集中、最完好的地区。"},

{ "南山森林公园", "南山森林公园是省级森林公园，总面积约 1500 公顷。"},

{ "镇江醋文化博物馆", "镇江醋文化博物馆是国内首个专业性醋文化主题博物馆。"},

{ "茅山景区", "茅山是中国道教名山，道教上清派的发源地。"},

{ "江苏茶博园", "江苏茶博园是集茶叶生产、科技示范、文化展示、旅游观光于一体的综合性园区。"},

{ "城上村遗址", "城上村遗址是新石器时代至西周时期的古文化遗址。"}

};

static char defaultIntro[] = "该景点是镇江著名的旅游景点，具有丰富的历史文化内涵和优美的自然风光。";

int mapSize = sizeof(introMap) / sizeof(introMap[0]);

```
for ( i = 0; i < mapSize; ++i) {
    if (strcmp(introMap[i].name, spotName) == 0) {
        return introMap[i].description;
    }
}
```

return defaultIntro;

}

void FindScenicSpot(ALGraph* G) {
 printf("\n=== 景点查找与路径规划 ===\n");

```
if (G->vexnum == 0) {
    printf("请先创建图! \n");
    return;
}
```

```
char spotName[50];
printf("请输入要查找的景点名称: ");
getchar();
fgets(spotName, sizeof(spotName), stdin);
spotName[strlen(spotName)] = '\0';
```

int spotIndex = LocateVexALG(G, spotName);

```
if (spotIndex == -1) {
```

```

        printf("× 景点 \"%s\" 不存在于图中! \n", spotName);
        return;
    }

    printf("\n√ 景点 \"%s\" 存在! \n", spotName);
    printf("    编号: %d\n", G->vertices[spotIndex].id);
    printf("    类型: ");
    switch(G->vertices[spotIndex].type) {
        case 'h': printf("人文类景点"); break;
        case 'e': printf("自然类景点"); break;
        case 'b': printf("双属性景点"); break;
        default: printf("未知类型"); break;
    }
    printf("\n");

    int jiangsuUnivIndex = LocateVexALG(G, "江苏大学");

    if (jiangsuUnivIndex == -1) {
        printf("× 图中没有找到江苏大学, 无法计算路径.\n");
        return;
    }

    double* dist = (double*)malloc(sizeof(double) * G->vexnum);
    int* prev = (int*)malloc(sizeof(int) * G->vexnum);
    Dijkstra(G, jiangsuUnivIndex, dist, prev);

    if (dist[spotIndex] >= INF) {
        printf("    从江苏大学到%s 无可达路径! \n", spotName);
        free(dist);
        free(prev);
        return;
    }

    printf("\n 从江苏大学到%s 的最短路径规划: \n", spotName);

    int* pathStack = (int*)malloc(sizeof(int) * G->vexnum);
    int stackTop = -1;
    int current = spotIndex;
    while (current != -1) {
        pathStack[++stackTop] = current;
        current = prev[current];
    }

    printf("    路径: ");

```

```

int first = 1;
double totalDistance = dist[spotIndex];

while (stackTop >= 0) {
    int vertex = pathStack[stackTop--];
    if (!first) printf(" -> ");
    printf("%s", G->vertices[vertex].data);
    first = 0;
}

printf("\n 总距离: %.11f km\n", totalDistance);

double estimatedTime = totalDistance / 30.0 * 60.0;
printf(" 预计时间: %.01f 分钟\n", estimatedTime);

printf("\n 景点介绍: \n");
printf(" %s\n", GetScenicIntroduction(spotName));

free(dist);
free(prev);
free(pathStack);
}

int main() {
    ALGraph G = {0};
    int choice = 0;
    int graphCreated = 0;

    do {
        printf("\n=== 镇江景点图管理系统 ===\n");
        printf("1. 创建连通镇江景点图\n");
        printf("2. 显示简洁图结构\n");
        printf("3. 构建同属性景点子图\n");
        printf("4. 查找景点、显示信息并规划路径\n");
        printf("5. 退出\n");
        printf("当前状态: %s\n", graphCreated ? "已创建图" : "未创建图");
        printf("请选择操作: ");
        scanf("%d", &choice);

        switch(choice) {

            case 1: {
                CreateConnectedZhenjiangGraph(&G);
                PrintGraph(&G);
            }
        }
    } while (choice != 5);
}

```

```

        graphCreated = 1;
        break;
    }

    case 2: {
        if (!graphCreated) {
            printf("请先创建图! \n");
            break;
        }
        PrintGraphSimple(&G);
        break;
    }

    case 3: {
        if (!graphCreated) {
            printf("请先创建图! \n");
            break;
        }
        char type;
        printf("请输入景点类型(h:人文, e:自然, b:双属性): ");
        getchar();
        scanf("%c", &type);
        PrintByType(&G, type);
        BuildSameAttributeSubgraph(&G);
        break;
    }

    case 4: {
        if (!graphCreated) {
            printf("请先创建图! \n");
            break;
        }
        FindScenicSpot(&G);
        break;
    }

    case 5: {
        printf("程序退出。 \n");
        break;
    }

    default: {
        printf("无效选择, 请重新选择。 \n");
        break;
    }
}

if (choice != 5) {

```



```

        printf("\n 操作完成，按回车键继续...");
        getchar();
        getchar();
    }

    while (choice != 5);

    if (graphCreated) {
        DestroyGraph(&G);
    }

    return 0;
}

```

四、上机结果及体会

1、 实际完成的情况说明（完成的功能，支持的数据类型等）：

1. 创建一个连通图 2. 将这个联通图进一步具体化，变为镇江大型景点的图 3. 将同一个类型的子图构建为连通图并且进行遍历 4. 根据给定的景点，查找出从江苏大学出发到给定景点的最短路径，并且输出该景点的介绍

2、 程序的性能分析

图的创建的时间复杂度为 $O(V+E)$ （顶点数量+边的数量），空间复杂度为 $O(V+E)$ （顶点数量+边的数量）

图顶点的查找时间复杂度为 $O(V)$ （顶点数量），空间复杂度为 $O(1)$

Dijkstra 算法时间复杂度为 $O(V^2+E)$ （顶点数量²+边的数量），空间复杂度为 $O(V+E)$ （顶点数量+边的数量）

深度遍历时间复杂度为 $O(V+E)$ （顶点数量+边的数量），空间复杂度为 $O(V)$ （顶点数量）

3、打印程序运行时的初值和运行结果，画出相应的图示

```

=== 镇江景点图管理系统 ===
1. 创建连通镇江景点图
2. 显示简洁图结构
3. 构建同属性景点子图
4. 查找景点、显示信息并规划路径
5. 退出
当前状态：已创建图
请选择操作：2

===== 简洁图结构信息（邻接表） =====
顶点数量：26，边数量：19
-----
城上村遗址 -> 邻接点：茅山景区(20.0km)，访仙镇(10.9km)，永安陵石刻(4.8km)
镇江醋文化博物馆 -> 邻接点：西津渡古街(5.5km)，镇江站南广场(4.9km)，南山森林公园(7.4km)
永安陵石刻 -> 邻接点：修安陵石刻(1.1km)，城上村遗址(4.8km)
修安陵石刻 -> 邻接点：访仙镇(6.3km)，永安陵石刻(1.1km)

```

=== 镇江景点图管理系统 ===

1. 创建连通镇江景点图
2. 显示简洁图结构
3. 构建同属性景点子图
4. 查找景点、显示信息并规划路径
5. 退出

当前状态：未创建图

请选择操作：1

=== 创建带权重的镇江景点图 ===

带权重的镇江景点图创建完成！

包含 26 个顶点和 18 条边

检测到 2 个连通分量，正在添加连接边...

添加连接边：苏宁广场 <-> 茅山景区（权重：20.0km）

添加连接边：茅山景区 <-> 城上村遗址（权重：20.0km）

连通图创建完成！

添加了2条连接边，使整个图连通。

最终连通分量数量：1

√ 图是连通图

===== 图结构信息（邻接表） =====

顶点数量：26，边数量：19

=== 镇江景点图管理系统 ===

1. 创建连通镇江景点图
2. 显示简洁图结构
3. 构建同属性景点子图
4. 查找景点、显示信息并规划路径
5. 退出

当前状态：已创建图

请选择操作：3

请输入景点类型(h:人文, e:自然, b:双属性): h

=== 人文类景点（共26个景点） ===

1. 城上村遗址（编号:1, 索引:0）
2. 镇江醋文化博物馆（编号:2, 索引:1）
3. 永安陵石刻（编号:3, 索引:2）
4. 修安陵石刻（编号:4, 索引:3）
5. 访仙镇（编号:5, 索引:4）
6. 吕城镇（编号:6, 索引:5）
7. 金山寺（编号:7, 索引:6）
8. 北固楼（编号:8, 索引:7）
9. 西津渡古街（编号:9, 索引:8）
10. 焦山定慧寺（编号:10, 索引:9）
11. 甘露寺铁塔（编号:11, 索引:10）
12. 镇江天主教堂（编号:12, 索引:11）
13. 铁瓮城遗址（编号:13, 索引:12）
14. 镇江体育会展中心（编号:14, 索引:13）
15. 镇江站南广场（编号:15, 索引:14）
16. 句容余坤开元大酒店（编号:16, 索引:15）
17. 苏宁广场（编号:17, 索引:16）
18. 江苏大学（编号:18, 索引:17）
19. 镇江火车站（编号:19, 索引:18）
20. 镇江市政府（编号:20, 索引:19）

总计：20 个人文类景点

=== 构建同属性景点子图 ===

操作完成，按回车键继续...

=== 镇江景点图管理系统 ===

1. 创建连通镇江景点图
2. 显示简洁图结构
3. 构建同属性景点子图
4. 查找景点、显示信息并规划路径
5. 退出

当前状态：已创建图

请选择操作：4

=== 景点查找与路径规划 ===

请输入要查找的景点名称：茅山景区

√ 景点 "茅山景区" 存在！

编号：25

类型：双属性景点

从江苏大学到茅山景区的最短路径规划：

路径：江苏大学 -> 镇江市政府 -> 苏宁广场 -> 茅山景区

总距离：27.9 km

预计时间：56 分钟

景点介绍：

茅山是中国道教名山，道教上清派的发源地。

4、上机过程中出现的问题及其解决方案

问题：这些景点过于分散，不知道怎么用图表示？

解决方法：设定阈值 20km，小于 20km 的就用边连接起来，如果最后的图不连通，那么就用一条权重为 20 的边将两个图最近的顶点连接起来，这样做符逻辑

5、程序中可以改进的地方说明

1. 有些函数算法复杂，通过改进可以减少空间复杂度与时间复杂度

2. 景点介绍可以另外存放，不用一直放在一个文件中

6、程序中可以扩充的功能及设计实现假想：

1. 实时交通指引：整合公交、地铁、打车数据，支持“起点-终点”路线规划（如镇江火车站到茅山景区），解决出行痛点。

2. 生活服务整合：覆盖美食（镇江肴肉、锅盖面）、酒店（茅山景区民宿）、活动（西津渡音乐节）预订，满足全场景需求。

3. 文化传播功能：通过图文、视频介绍镇江历史（如“镇江的起源”）、非遗（恒顺香醋制作技艺），强化文化体验。

4. 互动社交功能：支持游客评论、照片分享（如西津渡古街照片），鼓励攻略征集（如“三天两夜镇江游攻略”），增强用户粘性。

5. 应急与安全功能：提供一键求助（景区工作人员/报警）、安全提示（如茅山山路陡峭提醒），保障游客权益。

7、收获及体会：

这次上机让我体会到对于具体问题抽象分析的重要性，比如构建图这一步，我想了许久，如何将一个具体的例子抽象出来，并且形成一个数据结构呢？后来想到，图有两个指标，一个是点，一个是边，点很好定义，即是景点本身，但是边呢？若是形成一个完全连通图 K_n ，那么图必然会相当复杂，于是我就想到用一个阈值来定义两条边是否相连。但是这样一来，一些顶点之间便不连通了，这是非常不好的一件事，于是我就将两点定义成可以连通的，不过他们之间的距离比他们到团内部的顶点所有的距离都要大，这样就近似的解决了这样一个问题。

不过由于资料有限，我只能计算两点之间的直线距离，这个实际操作起来显然是不可能的，如果将镇江市的地图上面打上足够多的点当作中间的顶点，那么道路会曲折一些，也会更加接近实际情况，而且这样有助于分析路径。不过这样一来，图的顶点数与边数会变得无比庞大，做测试数据时也会非常复杂。

同时，特别感谢道格拉斯先生写的图论导引，让我能够更好的描述自己的观点

五、参考文献

BrilliantWilliam 数据结构——图的基本操作实现 CSDN 博客

https://blog.csdn.net/weixin_50961321/article/details/121663893

游子吟 镇江十大著名景点 知乎 [镇江十大著名景点 - 知乎](#)

Douglas B. West 《图论导引》

题目三： 限时算 24 点竞赛游戏

一、需求分析

- 1、 运行环境（软、硬件环境）：一般的 windows7 及以上系统，C98 环境（如 Devc++，至少需要包含 <stdio.h>、<stdlib.h>、<time.h>、<string.h>、<math.h>、<ctype.h>等标准库）
- 2、 程序所实现的功能
(1) 生成 4 个 10 以内的随机数 (2) 计算所给表达式的值 (3) 对选手的得分进行排序
- 3、 程序的输入：输入表达式，为字符串，其中仅仅能包含（已经给定的）数字，运算符，括号，以及求助运算符‘?’ 还有放弃运算符‘=’
- 4、 程序的输出：输出数字%d，字符串%s，用来引导玩家完成指定任务的游戏
- 5、 测试数据：无

二、设计说明

1、 算法设计的思想：

这个系统的主要难点有两个，一个是“如何计算表达式”，另外一个“如何计时”，第一个难点的攻克我用到了两个数据结构，一个是“树”，另外一个“栈”，由于有括号的存在，计算的优先级不是那么固定，于是就用栈先代为储存，从而构建表达式树，最后计算表达式树就可以了，其中，表达式树的构建是除了叶子节点是数以外，其他的节点都是运算符，而计算表达式树则是一个深度遍历的过程。

对于第二个难点，我在 CSDN 上面查找了诸多资料，最后选择了一个最简单而且通用的函数，，用 `time_t start_time = time(NULL)` 记录开始时间，用 `difftime(time(NULL), start_time)` 计算时间的差值。

2、 主要的数据结构设计说明

使用 typedef 定义了一个抽象数据类型：二叉树链表 struct BiTNode 定义如下：

该结构体包含三个结构体成员：

1.char data 为树该节点的元素

2.struct BiTNode *lchild 为指向树（队列）左子树的指针

3.struct BiTNode *rchild 为指向树（队列）右子树的指针

BiTNode 为 struct BiTNode 的简称，而 *BiTree 为指向 struct BiTNode 的指针类型

```
typedef struct BiTNode {  
    char data;  
    struct BiTNode *lchild;  
    struct BiTNode *rchild;  
}
```

```
BiTNode, *BiTree;
```

使用 typedef 定义了抽象数据类型栈 struct sta。

其中，栈 struct sta 定义如下：

该结构体包含三个结构体成员：

4. SElemType *base 为指向栈底的指针

5. SElemType *top 为指向栈顶的指针

6. int stacksize 为整个栈中元素的个数

7. SqStack 为 struct sta 的简称。

```
typedef struct sta{
```

```
SElemType *base;
```

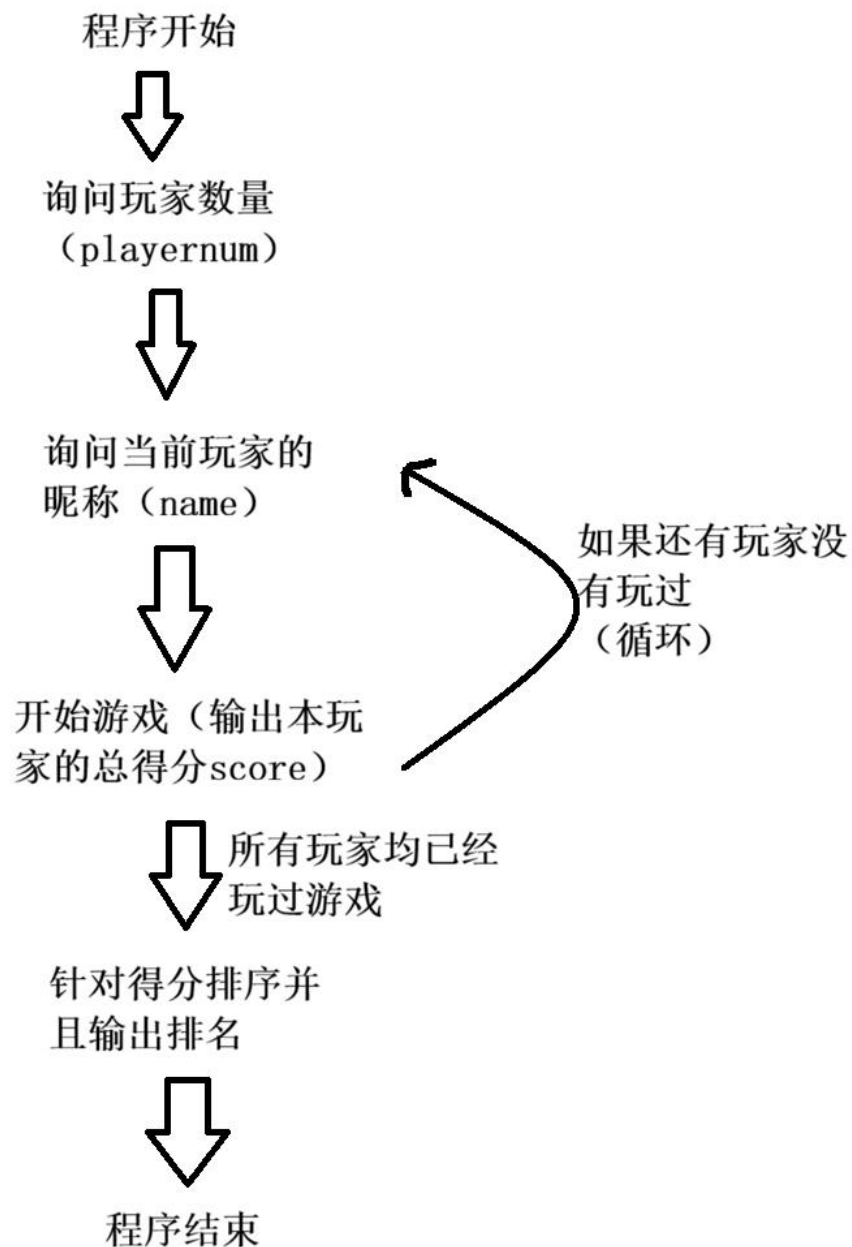
```
SElemType *top;
```

```
int stacksize;
```

```
}SqStack;
```

这边的栈元素为树的节点

3、 程序的主要流程图：



4、 程序的主要模块，要求对主要流程图出现的模块进行说明

开始游戏：玩家会依次完成 10 道题，可以有“完成”“错误”“放弃”“看答案”“超时”五种情况，其中，只有“完成”才能有相应的分数，10 道题目依次累计总得分，并且保存在玩家的结构体里面，等待全部完成后的排序。

针对得分排序并且输出排名：根据玩家结构体中的得分为标签，进行排序，最后输出所有玩家的得分。

5、程序的主要函数及其伪代码说明 (不需要完整的代码)

```
int ValidateNumbersUsed(BiTree root, int nums[4]) {
    int collect1[4] = {0};
    count = 0
    DFS(root, collect1, count)
    If count !=4;
        Return false
    For(i=0;i<2;i++)
        For(j=0;j<2-i;j++)
            If(collect1[j]>collect1[j+1])
                Swap(collect1[j],collect1[j+1]);
    Collect2 = nums
    For(i=0;i<2;i++)
        For(j=0;j<2-i;j++)
            If(collect2[j]>collect2[j+1])
                Swap(collect2[j],collect2[j+1]);
    For(i=0;i<3;i++)
        If(collect1[i]!=collect2[i])            return false
    Return true
BiTree CreateExpTree(const char* exp, int* error, int nums[4]) {
    InitSack EXPT
    InitSack OPTR
    Push( '#', OPTR)
    i = 0
    length = strlen(exp)
    while i < length:
        ch = exp[i]
        if isnum(ch):
            value = 0
            while i < length && isnum(exp[i]) :
                value = value * 10 + (exp[i] - '0')
                i += 1
            i -= 1
            BiTreeNode* node = CreateExpTreeNode(NULL, NULL, value);
            PushTree(&EXPT, node);
        else if ch == '(':
            Push(ch,OPTR );
        else if ch == ')':
            while OPTR != '(':
```

```

        op = Pop (OPTR )
        b = Pop (EXPT )
        a = Pop (EXPT )
        if !a || !b :
            error = 1
            Return;
        BiTreeNode* node = CreateExpTreeNode(a, b, op);
        PushTree(&EXPT, node);
    else if ch ==(+, -, *, /):
        while !IsEmpty(OPTR) && 栈顶运算符优先级 >= 当前运算符优先级:
            op = Pop (OPTR )
            b = Pop (EXPT )
            a = Pop (EXPT )
            if !a || !b :
                error = 1
                Return;
            BiTreeNode* node = CreateExpTreeNode(a, b, op);
            PushTree(&EXPT, node);
            PushChar(&OPTR, ch);
    else if ch ==' ' :
    else:
        error= 1
        Return ;
    i += 1
While Pop() !=' #' :
    op = Pop (OPTR )
    b = Pop (EXPT )
    a = Pop (EXPT )
    if !a || !b :
        error = 1
        Return ;
    BiTreeNode* node = CreateExpTreeNode(a, b, op);
    PushTree(&EXPT, node);
root = POP(EXPT )
if !IsEmpty(EXPT) :
    error = 1
    Free(root)
    Return ;
error = 0
Return root

```

三、源代码

1. 主函数及所有子函数(完整的代码)

```
#include<bits/stdc++.h>
#define MAX_SIZE 100
#define MAX_NAME_LEN 50
#define MAX_PLAYER_NUM 3 //游戏人数
#define MAX_TIME 180
#define TOTAL_QUESTIONS 2 //总问题数
#define NUM_OPS 4
#define NUM_STRUCTURES 5
#define EPS 1e-6

typedef struct Node {
    int data;
    struct Node *leftChild;
    struct Node *rightChild;
} BiTreeNode, *BiTree;

typedef struct {
    BiTreeNode* data[MAX_SIZE];
    int top;
} TreeStack;

typedef struct {
    char data[MAX_SIZE];
    int top;
} CharStack;

void GenerateRandomNumbers(int nums[4]) {
    int i;
    for (i = 0; i < 4; i++) {
        nums[i] = rand() % 10 + 1;
    }
}

typedef struct {
    char name[MAX_NAME_LEN];
    int score;
} Player;

void ClearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
```



```
}
```

```
BiTreeNode* CreateExpTreeNode(BiTreeNode *a, BiTreeNode *b, int value) {  
    BiTreeNode* T = (BiTreeNode*)malloc(sizeof(BiTreeNode));  
    T->data = value;  
    T->leftChild = a;  
    T->rightChild = b;  
    return T;  
}
```

```
int GetValue(char theta, int a, int b) {  
    switch (theta) {  
        case '+': return a + b;  
        case '-': return a - b;  
        case '*': return a * b;  
        case '/':  
            if (b == 0) {  
                return 0;  
            }  
            return a / b;  
        default: return 0;  
    }  
}
```

```
int EvaluateExpressionTree(BiTree T) {  
    if (!T->leftChild && !T->rightChild) {  
        return T->data;  
    }  
    int leftValue = EvaluateExpressionTree(T->leftChild);  
    int rightValue = EvaluateExpressionTree(T->rightChild);  
    return GetValue((char)T->data, leftValue, rightValue);  
}
```

```
int GetPriority(char op) {  
    switch (op) {  
        case '+':  
        case '-': return 1;  
        case '*':  
        case '/': return 2;  
        case '(':  
        case ')': return 0;  
        case '#': return -1;  
        default: return 0;  
    }  
}
```

```

}

void InitTreeStack(TreeStack* stack) {
    stack->top = -1;
}

void InitCharStack(CharStack* stack) {
    stack->top = -1;
}

int IsTreeStackEmpty(TreeStack* stack) {
    return stack->top == -1;
}

int IsCharStackEmpty(CharStack* stack) {
    return stack->top == -1;
}

void PushTree(TreeStack* stack, BiTreeNode* node) {
    if (stack->top < MAX_SIZE - 1) {
        stack->data[++stack->top] = node;
    }
}

void PushChar(CharStack* stack, char ch) {
    if (stack->top < MAX_SIZE - 1) {
        stack->data[++stack->top] = ch;
    }
}

BiTreeNode* PopTree(TreeStack* stack) {
    if (!IsTreeStackEmpty(stack)) {
        return stack->data[stack->top--];
    }
    return NULL;
}

char PopChar(CharStack* stack) {
    if (!IsCharStackEmpty(stack)) {
        return stack->data[stack->top--];
    }
    return '\0';
}

```

```

BiTreeNode* TopTree(TreeStack* stack) {
    if (!IsTreeStackEmpty(stack)) {
        return stack->data[stack->top];
    }
    return NULL;
}

char TopChar(CharStack* stack) {
    if (!IsCharStackEmpty(stack)) {
        return stack->data[stack->top];
    }
    return '\\0';
}

void CollectNumbersFromTree(BiTree root, int collected[4], int* count) {
    if (root == NULL) return;

    CollectNumbersFromTree(root->leftChild, collected, count);
    CollectNumbersFromTree(root->rightChild, collected, count);

    if (!root->leftChild && !root->rightChild) {
        collected[*count] = root->data;
        (*count)++;
    }
}

int ValidateNumbersUsed(BiTree root, int nums[4]) {
    int collected[4] = {0};
    int count = 0;
    int i, j;

    CollectNumbersFromTree(root, collected, &count);

    if (count != 4) {
        return 0;
    }

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3 - i; j++) {
            if (collected[j] > collected[j+1]) {
                int temp = collected[j];
                collected[j] = collected[j+1];
                collected[j+1] = temp;
            }
        }
    }
}

```

```

    }
}

int sorted_nums[4];
for (i = 0; i < 4; i++) {
    sorted_nums[i] = nums[i];
}

for (i = 0; i < 3; i++) {
    for (j = 0; j < 3 - i; j++) {
        if (sorted_nums[j] > sorted_nums[j+1]) {
            int temp = sorted_nums[j];
            sorted_nums[j] = sorted_nums[j+1];
            sorted_nums[j+1] = temp;
        }
    }
}

for (i = 0; i < 4; i++) {
    if (collected[i] != sorted_nums[i]) {
        return 0;
    }
}

return 1;
}

void FreeExpressionTree(BiTree root) {
    if (root == NULL) return;
    FreeExpressionTree(root->leftChild);
    FreeExpressionTree(root->rightChild);
    free(root);
}

BiTree CreateExpTree(const char* exp, int* error, int nums[4]) {
    TreeStack EXPT;
    CharStack OPTR;
    InitTreeStack(&EXPT);
    InitCharStack(&OPTR);
    PushChar(&OPTR, '#');

    int i = 0;
    int exp_len = strlen(exp);

```

```

while (i < exp_len) {
    char ch = exp[i];
    if (isdigit(ch)) {
        int value = 0;
        while (i < exp_len && isdigit(exp[i])) {
            value = value * 10 + (exp[i] - '0');
            i++;
        }
        i--;

        BiTreeNode* node = CreateExpTreeNode(NULL, NULL, value);
        PushTree(&EXPT, node);
    }
    else if (ch == '(') {
        PushChar(&OPTR, ch);
    }
    else if (ch == ')') {
        while (TopChar(&OPTR) != '(') {
            char op = PopChar(&OPTR);
            BiTreeNode* b = PopTree(&EXPT);
            BiTreeNode* a = PopTree(&EXPT);
            if (a == NULL || b == NULL) {
                *error = 1;
                return NULL;
            }
            PushTree(&EXPT, CreateExpTreeNode(a, b, op));
        }
        PopChar(&OPTR);
    }
    else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
        while (!IsCharStackEmpty(&OPTR) &&
            GetPriority(TopChar(&OPTR)) >= GetPriority(ch)) {
            char op = PopChar(&OPTR);
            BiTreeNode* b = PopTree(&EXPT);
            BiTreeNode* a = PopTree(&EXPT);
            if (a == NULL || b == NULL) {
                *error = 1;
                return NULL;
            }
            PushTree(&EXPT, CreateExpTreeNode(a, b, op));
        }
        PushChar(&OPTR, ch);
    }
    else if (ch == ' ') {

```

```

    }
    else {
        *error = 1;
        return NULL;
    }
    i++;
}
while (TopChar(&OPTR) != '#') {
    char op = PopChar(&OPTR);
    BiTreeNode* b = PopTree(&EXPT);
    BiTreeNode* a = PopTree(&EXPT);
    if (a == NULL || b == NULL) {
        *error = 1;
        return NULL;
    }
    PushTree(&EXPT, CreateExpTreeNode(a, b, op));
}
BiTree root = PopTree(&EXPT);
if (!IsTreeStackEmpty(&EXPT)) {
    *error = 1;
    if (root != NULL) {
        FreeExpressionTree(root);
    }
    return NULL;
}

*error = 0;
return root;
}

```

```

double applyOp(double a, double b, char op) {
    switch (op) {
        case '+':
            return a + b;
        case '-':
            return a - b;
        case '*':
            return a * b;
        case '/':
            if (fabs(b) < EPS) {
                return 1e9;
            }
            return a / b;
        default:

```

```

        return 0.0;
    }
}

double calculate(int nums[4], char ops[3], int structure) {
    double a = (double)nums[0];
    double b = (double)nums[1];
    double c = (double)nums[2];
    double d = (double)nums[3];
    double ab, bc, cd, abc, bcd, ab_cd;

    switch (structure) {
        case 0:
            ab = applyOp(a, b, ops[0]);
            abc = applyOp(ab, c, ops[1]);
            return applyOp(abc, d, ops[2]);
        case 1:
            bc = applyOp(b, c, ops[1]);
            abc = applyOp(a, bc, ops[0]);
            return applyOp(abc, d, ops[2]);
        case 2:
            bc = applyOp(b, c, ops[1]);
            bcd = applyOp(bc, d, ops[2]);
            return applyOp(a, bcd, ops[0]);
        case 3:
            cd = applyOp(c, d, ops[2]);
            bcd = applyOp(b, cd, ops[1]);
            return applyOp(a, bcd, ops[0]);
        case 4:
            ab = applyOp(a, b, ops[0]);
            cd = applyOp(c, d, ops[2]);
            return applyOp(ab, cd, ops[1]);
        default:
            return 0.0;
    }
}

void exprString(int nums[4], char ops[3], int structure, char *buf) {
    switch (structure) {
        case 0:
            sprintf(buf, "((%d%c%d)%c%d)%c%d", nums[0], ops[0], nums[1], ops[1],
nums[2], ops[2], nums[3]);
            break;
        case 1:

```

```

        sprintf(buf, "(%d%c(%d%c%d))%c%d", nums[0], ops[0], nums[1], ops[1],
nums[2], ops[2], nums[3]);
        break;
    case 2:
        sprintf(buf, "%d%c((%d%c%d)%c%d)", nums[0], ops[0], nums[1], ops[1],
nums[2], ops[2], nums[3]);
        break;
    case 3:
        sprintf(buf, "%d%c(%d%c(%d%c%d))", nums[0], ops[0], nums[1], ops[1],
nums[2], ops[2], nums[3]);
        break;
    case 4:
        sprintf(buf, "(%d%c%d)%c(%d%c%d)", nums[0], ops[0], nums[1], ops[1],
nums[2], ops[2], nums[3]);
        break;
    }
}

```

```

void swap_int(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```

void permute(int arr[4], int start, char ops[NUM_OPS]) {
    int i, j, k, s;
    if (start == 4) {
        char op3[3];
        for (i = 0; i < NUM_OPS; i++) {
            for (j = 0; j < NUM_OPS; j++) {
                for (k = 0; k < NUM_OPS; k++) {
                    op3[0] = ops[i];
                    op3[1] = ops[j];
                    op3[2] = ops[k];
                    for (s = 0; s < NUM_STRUCTURES; s++) {
                        double result = calculate(arr, op3, s);
                        if (fabs(result - 24.0) < EPS) {
                            char buf[100];
                            exprString(arr, op3, s, buf);
                            printf("%s\n", buf);
                        }
                    }
                }
            }
        }
    }
}

```



```

    }
} else {
    for (i = start; i < 4; i++) {
        swap_int(&arr[start], &arr[i]);
        permute(arr, start + 1, ops);
        swap_int(&arr[start], &arr[i]);
    }
}
}

void solve(int nums[4]) {
    char ops[NUM_OPS] = {'+', '-', '*', '/'};
    int arr[4];
    int i;
    for (i = 0; i < 4; i++) {
        arr[i] = nums[i];
    }
    permute(arr, 0, ops);
}

int Play24Game() {
    int total_score = 0;
    int question_num = 0;
    int i;

    srand(time(NULL));

    while (question_num < TOTAL_QUESTIONS) {
        question_num++;
        printf("\n=== 第%d 题 =====进度%d/%d====\n",
question_num,question_num,TOTAL_QUESTIONS);
        int nums[4];
        GenerateRandomNumbers(nums);
        printf("数字: %d %d %d %d\n", nums[0], nums[1], nums[2], nums[3]);
        time_t start_time = time(NULL);
        int attempts = 0;
        int answered = 0;
        while (!answered) {
            double elapsed = difftime(time(NULL), start_time);
            if (elapsed >= MAX_TIME) {
                printf("? 时间到! 本题未完成\n");
                printf("超时%d 秒, 本题得分: 0\n", (int)elapsed);
                answered = 1;
                break;
            }
        }
    }
}

```

```

}

printf("\n 剩余时间: %d 秒 | 尝试次数: %d\n",
      MAX_TIME - (int)elapsed, attempts);
printf("请输入表达式（直接输入表达式即可）: ");
char exp[MAX_SIZE];
if (scanf("%99s", exp) != 1) {
    printf("输入错误, 请重试! \n");
    ClearInputBuffer();
    continue;
}

else if (strcmp(exp, "=") == 0) {
    printf("放弃当前题目, 进入下一题\n");
    answered = 1;
    break;
}
else if (strcmp(exp, "?") == 0) {
    solve(nums);
    break;
}
attempts++;
int error = 0;
BiTree root = CreateExpTree(exp, &error, nums);

if (error) {
    printf("表达式错误, 请检查格式! \n");
    if (root != NULL) {
        FreeExpressionTree(root);
    }
    continue;
}
int valid_nums = ValidateNumbersUsed(root, nums);
if (!valid_nums) {
    printf(" 必须使用全部给定的 4 个数字, 每个数字只能使用一次! \n");
    FreeExpressionTree(root);
    continue;
}
int result = EvaluateExpressionTree(root);
if (fabs(result - 24) < EPS) {
    elapsed = difftime(time(NULL), start_time);
    int score = (int)(100 * (MAX_TIME - elapsed) / MAX_TIME);
    score = score < 0 ? 0 : score;
}

```

```

        printf(" 正确! 结果 = 24\n");
        printf("用时: %.1f 秒 | 尝试次数: %d\n", elapsed, attempts);
        score += 180 - (int)elapsed;
        printf("本题得分: %d 分\n", score);

        total_score += score;
        FreeExpressionTree(root);
        answered = 1;
    } else {
        printf(" 结果错误! 计算结果 = %d (不是 24) \n", result);
        FreeExpressionTree(root);
    }
}

printf("当前总分: %d\n", total_score);
}

printf("\n===== 游戏结束 =====\n");
printf("完成题目: %d 道\n", TOTAL_QUESTIONS);
printf("最终得分: %d 分\n", total_score);
printf("平均每题得分: %.1f 分\n", (float)total_score / TOTAL_QUESTIONS);
return total_score;
}

int OneTry() {
    int m;
    printf("===== 24 点游戏 =====\n");
    printf("==游戏版本: 一共%d 道==\n", TOTAL_QUESTIONS);
    printf("规则: \n");
    printf("1. 系统会随机给出 4 个 1-10 的数字\n");
    printf("2. 你需要用+、-、*、/和括号组成表达式, 使结果等于 24\n");
    printf("3. 每道题限时%d 秒, 超过时间自动进入下一题\n", MAX_TIME);
    printf("4. 共有%d 道题, 每道题根据用时计分\n", TOTAL_QUESTIONS);
    printf("5. 输入 '=' 结束当前题目\n\n");
    printf("特别提醒: /为整除, 若放弃这一题, 则直接输入 '=';这里的括号为英文的括号! ;\n 输入 '?' 获取本题解的所有答案");
    m = Play24Game();
    return m;
}

void Swap(Player *p1, Player *p2) {
    Player temp;
    strcpy(temp.name, p1->name);
    temp.score = p1->score;

```

```

        strcpy(p1->name, p2->name);
        p1->score = p2->score;
        strcpy(p2->name, temp.name);
        p2->score = temp.score;
    }

void SelectionSort(Player arr[], int size) {
    int i, j, k;
    Player temp;
    for (i = 0; i < size - 1; i++) {
        if(arr[i].score == -1 && i < size - 2) continue;
        k = i;
        for (j = i + 1; j < size; j++) {
            if(arr[j].score == -1 && j < size - 1) continue;
            if (arr[j].score > arr[k].score) {
                k = j;
            }
        }
        if (k != i) {
            Swap(&arr[k], &arr[i]);
        }
    }
}

void printRank(Player arr[], int n) {
    int i;
    printf("====游戏结算排名====\n");
    for(i = 0; i < n; i++) {
        if(arr[i].score == -1) continue;
        printf("第%d名: %s\t得分:%d\n", i+1, arr[i].name, arr[i].score);
    }
}

int main() {
    int playernum;
    int i;
    Player team[MAX_PLAYER_NUM];
    for(i = 0; i < MAX_PLAYER_NUM; i++) {
        team[i].score = -1;
    }

    printf("请输入本局玩家数量: ");
    scanf("%d", &playernum);
    if(playernum>MAX_PLAYER_NUM)

```

```

    {printf("最大人数不能超过%d 人! ",MAX_PLAYER_NUM);
    printf("请重新输入本局玩家数量: ");
    scanf("%d", &playernum);}
    ClearInputBuffer();

    for(i = 0; i < playernum; i++) {
        printf("请输入当前玩家昵称: ");
        fgets(team[i].name, MAX_NAME_LEN, stdin);
        team[i].name[strcspn(team[i].name, "\n")] = '\0';
        team[i].score = OneTry();
        printf("本局玩家结束, 按任意键进行下一轮");
        getchar();getchar();
    }
    SelectionSort(team, playernum);
    printRank(team, playernum);
    printf("\n 游戏结束! 感谢参与! \n");
    return 0;
}

```

四、上机结果及体会

1、 实际完成的情况说明（完成的功能，支持的数据类型等）

完成的功能：计时+计算表达式的值+给出答案

数据类型：字符串

2、程序的性能分析，包括时空分析

由于该程序的规模固定，所以无论是空间复杂度还是时间复杂度均为 $O(1)$ 。

3、打印程序运行时的初值和运行结果，画出相应的图示

```

剩余时间：150秒 | 尝试次数：1
请输入表达式（直接输入表达式即可）：10*9+7+4
? 结果错误！计算结果 = 101（不是24）
? 时间到！本题未完成
超时188秒，本题得分：0
当前总分：0

```

回答错误显示

```
=== 第2题 ===  
数字：9 3 1 7  
  
剩余时间：180秒 | 尝试次数：0  
请输入表达式（直接输入表达式即可）：=  
放弃当前题目，进入下一题  
当前总分：0
```

跳过题目显示

```
=== 第1题 =====进度1/2=====  
数字：1 4 6 10  
  
剩余时间：180秒 | 尝试次数：0  
请输入表达式（直接输入表达式即可）：?  
表达式错误，请检查格式！  
  
剩余时间：162秒 | 尝试次数：1  
请输入表达式（直接输入表达式即可）：?  
((4-1)*10)-6  
(10*(4-1))-6  
当前总分：0
```

获取答案显示

```
=== 第3题 ===  
数字：3 10 3 5  
  
剩余时间：180秒 | 尝试次数：0  
请输入表达式（直接输入表达式即可）：(10-5+3)*3  
? 表达式错误，请检查格式！  
  
剩余时间：67秒 | 尝试次数：1  
请输入表达式（直接输入表达式即可）：(10-5+3)*3  
? 正确！结果 = 24  
用时：125.0秒 | 尝试次数：2  
本题得分：85分  
当前总分：85
```

回答正确显示

```
=====游戏结算排名=====  
第1名：bruce lee      得分：0  
第2名：yu             得分：0  
游戏结束！感谢参与！  
-----
```

结算显示

4、上机过程中出现的问题及其解决方案

问题：sleep 函数一般用于 mas 与 Linux 系统，不适用于 windows 的编译

解决方案：换用函数：Sleep(n * 1000)，故需要用到<Windows.h>库调用

问题：想要完成给出答案的过程太过于复杂，不好同时处理

解决方案：分成两个部分，一个是传统的问题，另外一个给出四个十以内的数字，生成值为 24 的表达式，先分别解决，然后分别封装，然后将他们合并到一块。

问题：在进行比较的时候，一些数字的尾数由于计算机的存储问题，没有办法完全一致导致错误

解决方案：设置一个波动范围，只要波动范围不超过 10^{-6} 那么就认为这个数字是相等的，给数字预留了很大的失误空间。

5、程序中可以改进的地方说明

有些生成的表达式如“1 1 1 10”无法通过表达式构建出一个值为 24 的表达式，可以在生成完显示之前先验证一下，是否存在答案

使用快速排序进行排序而并不是选择排序

记录玩家所完成的题目数量，在结算的时候输出

6、程序中可以扩充的功能及设计实现假想

记录已计算的数字组合和运算符排列，避免重复计算

在生成表达式树时直接应用优先级规则，减少括号结构的枚举量

开发成界面的形式，让用户在框内输入，并且实时提醒剩余时间

越难的题越加分，让比赛更公平

将两位挑战者放在一起，抢答后答对者加分，如果答错，则直接换下一题

7、收获及体会

在完成这个实验的时候，我深刻的体会到了“书到用时方恨少”，面对“计时系统”的设计，我毫无头绪，甚至在 CSDN 上面搜到的一些方法，不仅复杂，还不适用于 DEV++ 的环境，好在后来解决了。在构建表达式树的时候，终于用上了学过的知识，不过老师上课没有实操过，试了好久才试出来。

五、参考文献

南城花随雪.C 语言实现计时（附带源码 CSDN 博客

<https://katie.blog.csdn.net/article/details/145430230>

小辉_Super.C 语言小项目——计时器（倒计时+报警提示） CSDN 博客

<https://xiaohuisuper.blog.csdn.net/article/details/121347301>

Holmofy.C 语言计时函数 CSDN 博客

<https://holmofy.blog.csdn.net/article/details/76266484>

apcipot_rain.数据结构实验——基于二叉树的表达式求值算法

https://blog.csdn.net/xinghuayu_lin/article/details/145522836

飘飘历险记.十大经典排序算法（C 语言实现）

<https://blog.csdn.net/mwj327720862/article/details/80498455>

bili_73704135093.数据结构 | 习题 _ 二叉树求表达式的值 bilibili 视频

https://www.bilibili.com/video/BV1oz421y7Lz?vd_source=b07b4b083f9d64d9588551dfd9c6b5bf

题目四： 哈夫曼编码姓名增加传输效率

一、需求分析

- 1、 运行环境（软、硬件环境）：一般的 windows7 及以上系统，C98 环境（如 Devc++，至少需要包含 <stdio.h>、<stdlib.h>、<string.h> 等标准库）
- 2、 程序所实现的功能：对给定（固定）的成员进行哈夫曼编码，并且可以进行通过编码查找成员，或者通过成员（名称）查找哈夫曼编码，即编码与解码的过程。同时在输出的时候可以根据频率进行一次选择排序。
- 3、 程序的输入：可以输入成员的姓名%s 与哈夫曼编码%s，其中格式在计算机中的存储都为字符串格式。但是当输入选择进入的系统时输入的是一个整型数字而不是字符串。
- 4、 程序的输出：大多数以表格形式输出，只在单次查找某个人时以结构体的形式输出
- 5、 测试数据：形如{16, "赵", ""}的测试数据，一共 28 个。

二、设计说明

1、 算法设计的思想

（1）系统发展背景：

在苏超开展之际，球迷们纷纷为自己喜欢的球员应援，平台发现有一些球员的应援人数较多，于是开发了这个系统，将各个球员进行哈夫曼编码，降低数字传输量，在能够完整的传输球迷们的喜爱的同时，最大的缩减传输的损耗。

（2）哈夫曼编码的优越性

哈夫曼编码的优越性在于：长度短，加密性好，没有歧义，效率高，并且易于解码等特点，而且在如今这个网络发达，大家均可产生网络消息的时代，高效率的通讯迫在眉睫。而且哈夫曼树的构建时只需要静态存储，不需要动态分配内存，空间复杂度相对固定。

（3）哈夫曼编码的设计

当用 n 个结点（都做叶子结点且都有各自的权值）试图构建一棵树时，如果构建的这棵树的带权路径长度最小，称这棵树为“最优二叉树”，有时也叫“赫夫曼树”或者“哈夫曼树”。在构建哈夫曼树时，要使树的带权路径长度最小，只需要遵循一个原则，那就是：权重越大的结点离树根越近。在图 1 中，因为结点 a 的权值最大，所以理应直接作为根结点的孩子结点。

哈夫曼树的构建过程：

1.在 n 个权值中选出两个最小的权值，对应的两个结点组成一个新的二叉树，且新二叉树的根结点的权值为左右孩子权值的和；

2.在原有的 n 个权值中删除那两个最小的权值，同时将新的权值加入到 $n-2$ 个权值的行列中，以此类推；

3.重复 1 和 2，直到所有的结点构建成了一棵二叉树为止，这棵树就是哈夫曼树。□

（4）从哈夫曼树到哈夫曼编码

哈夫曼编码是在哈夫曼树的基础上实现的，具体实现方法是，从哈夫曼（二叉）树根节点出发，往左孩子走就标 0，往右孩子走就标 1，直到走到叶子节点为止，解码完成。编码就是解码的逆过程，从叶子节点出发，直到走到根节点为止。

2、 主要的数据结构设计说明

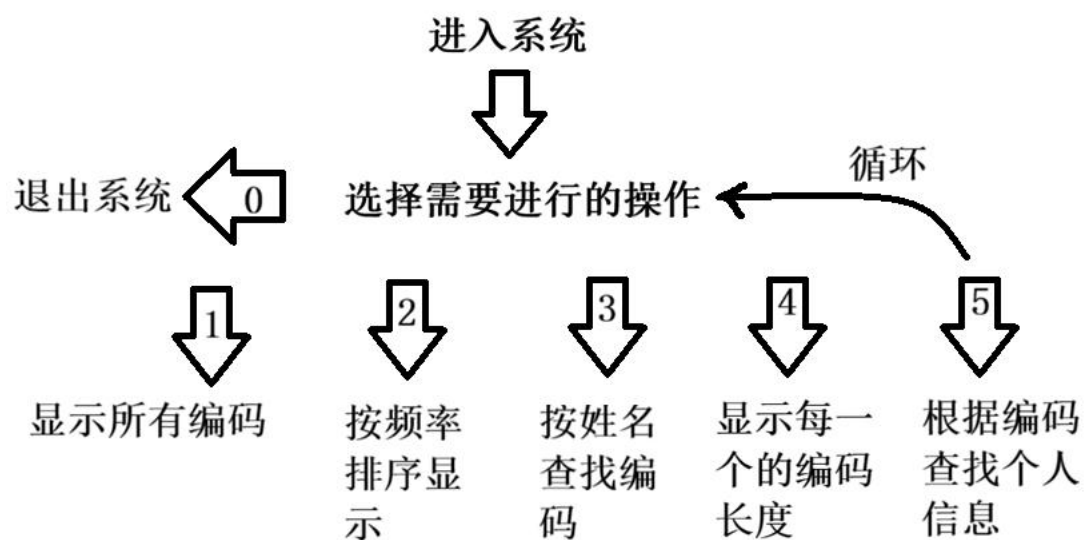
用数组生成一棵哈夫曼树的时候，我们用 typedef 定义一个结构体数据类型，结构体有四个成员变量——父节点，以及两个子节点（左孩子与右孩子），还有当前节点的权重。将其记为 HTNode，而 HuffmanTree 类型则为指向该节点的指针变量类型。

总体结构体定义如下：

```
typedef struct {  
    int weight;//当前节点的权重  
    int parent, left, right;//其父节点与子节点的下标  
} HTNode, *HuffmanTree;
```

在用如是结构体构造哈夫曼树时，首先，需要创建 $2n-1$ 个节点（ n 为给定数组的长度）构成 HTNode 的一个一维数组，然后依次选择，根据上述的构建哈夫曼树的方式构建。

3、 程序的主要流程图：



4、 程序的主要模块

(1) 显示所有编码：显示内置的所有编码与其对应球员的姓名。

(2) 按频率排序显示：将球员的姓名在应援中的出现频率从高到低排列，并且后面附带哈夫曼编码，哈夫曼编码从上到下依次边长。

(3) 按姓名查找编码：输入球员姓名，查找对应的哈夫曼编码，即模拟编码过程，输出哈夫曼编码。

(4) 显示编码长度：显示每个球员的哈夫曼编码长度。

(5) 根据编码查找个人信息：为“按姓名查找编码”的逆过程，即为解码过程，输出姓名等

5、 程序的主要函数及其伪代码说明

```
void Select(HuffmanTree HT, int end, int *s1, int *s2) {  
    i = 1  
    min1 = INFINITY  
    min2 = INFINITY  
    while i <= end && HT[i].parent != 0:  
        i++;  
    min1 = HT[i].weight  
    *s1 = i  
    i ++;
```

```

while i <= end:
    if HT[i].parent == 0 且 HT[i].weight < min1:
        min1 = HT[i].weight
        *s1 = i
    i ++
i = 1
while i <= end:
    if HT[i].parent == 0 && i != *s1:
        break
    i = i + 1
min2 = HT[i].weight
*s2 = i
i = i + 1
while i <= end:
    if HT[i].parent == 0 && i != *s1 && HT[i].weight < min2:
        min2 = HT[i].weight
        *s2 = i
    i ++;
}

void HuffmanCoding(HuffmanTree HT, HuffmanCode *HC, int n) {
    HC = 分配 (n+1) 个字符指针的数组
    cd = 分配 n 个字符的临时数组, 末尾置'\0'
    for i from 1 to n:
        start = n-1
        c = i
        f = HT[i].parent
        while f != 0:
            if HT[f].left == c:
                cd[start] = '0'
            else:
                cd[start] = '1'
            start -= 1
            c = f
            f = HT[f].parent
        HC[i] = 分配 (n - start) 个字符的空间
        for j from start to n-1:
            HC[i][j - start] = cd[j]
        HC[i][n - start - 1] = '\0'
    Free(cd);}

```

三、源代码

1. 主函数及所有子函数(完整的代码)

```
#include<bits/stdc++.h>
```

```

#define MAX_ID_LEN 20
#define MAX_NAME_LEN 20
typedef struct {
    int frequency;
    char name[MAX_NAME_LEN];
    char id[MAX_ID_LEN];
} admin;

typedef struct {
    int weight;
    int parent, left, right;
} HTNode, *HuffmanTree;
typedef char **HuffmanCode;
void Select(HuffmanTree HT, int end, int *s1, int *s2) {
    int i = 1;
    while (i <= end && HT[i].parent != 0) i++;
    int min1 = HT[i].weight;
    *s1 = i++;
    while (i <= end && HT[i].parent != 0) i++;
    int min2 = HT[i].weight;
    *s2 = i;
    if (min1 > min2) {
        int temp = min1; min1 = min2; min2 = temp;
        temp = *s1; *s1 = *s2; *s2 = temp;
    }
    for (int j = i + 1; j <= end; j++) {
        if (HT[j].parent != 0) continue;
        if (HT[j].weight < min1) {
            min2 = min1; *s2 = *s1;
            min1 = HT[j].weight; *s1 = j;
        } else if (HT[j].weight < min2) {
            min2 = HT[j].weight; *s2 = j;
        }
    }
}

void CreateHuffmanTree(HuffmanTree *HT, admin *admins, int n) {
    int m = 2 * n - 1;
    *HT = (HuffmanTree)malloc((m + 1) * sizeof(HTNode));
    for (int i = 1; i <= n; i++) {
        (*HT)[i].weight = admins[i - 1].frequency;
        (*HT)[i].parent = (*HT)[i].left = (*HT)[i].right = 0;
    }
    for (int i = n + 1; i <= m; i++) {

```

```

        (*HT)[i].weight = (*HT)[i].parent = (*HT)[i].left = (*HT)[i].right = 0;
    }
    for (int i = n + 1; i <= m; i++) {
        int s1, s2;
        Select(*HT, i - 1, &s1, &s2);
        (*HT)[s1].parent = (*HT)[s2].parent = i;
        (*HT)[i].left = s1;
        (*HT)[i].right = s2;
        (*HT)[i].weight = (*HT)[s1].weight + (*HT)[s2].weight;
    }
}

```

```

void HuffmanCoding(HuffmanTree HT, HuffmanCode *HC, int n) {
    *HC = (HuffmanCode)malloc((n + 1) * sizeof(char *));
    char *cd = (char *)malloc(n * sizeof(char));
    cd[n - 1] = '\0';
    for (int i = 1; i <= n; i++) {
        int start = n - 1;
        for (int c = i, f = HT[i].parent; f != 0; c = f, f = HT[f].parent) {
            cd[--start] = HT[f].left == c ? '0' : '1';
        }
        (*HC)[i] = (char *)malloc((n - start) * sizeof(char));
        for (int j = start; j < n; j++) {
            (*HC)[i][j - start] = cd[j];
        }
        (*HC)[i][n - start - 1] = '\0';
    }
    free(cd);
}

```

```

int HuffmanTreeCode(admin admins[], int n) {
    HuffmanTree HT = NULL;
    HuffmanCode HC = NULL;
    CreateHuffmanTree(&HT, admins, n);
    HuffmanCoding(HT, &HC, n);
    for (int i = 1; i <= n; i++) {
        strcpy(admins[i-1].id, HC[i]);
    }
    for (int i = 1; i <= n; i++) {
        free(HC[i]);
    }
    free(HC);
    free(HT);
    return 0;
}

```

```

}

void SortAdminsByFrequency(admin admins[], int n) {
    int i,j,k;
    for (i = 0; i < n-1; i++) {
        for (j = 0; j < n-i-1; j++) {
            if (admins[j].frequency < admins[j+1].frequency) {
                admin temp = admins[j];
                admins[j] = admins[j+1];
                admins[j+1] = temp;
            }
        }
    }
}

void Swap(admin *p1,admin *p2){
    admin temp;
    strcpy(temp.name, p1->name);
    temp.frequency = p1->frequency;
    strcpy(temp.id,p1->id);
    strcpy(p1->name, p2->name);
    p1->frequency = p2->frequency;
    strcpy(p1->id,p2->id);
    strcpy(p2->name, temp.name);
    p2->frequency = temp.frequency;
    strcpy(p2->id,temp.id);
}

void SelectionSort(admin arr[], int size) {
    int i, j, k;
    //admin temp;
    for (i = 0; i < size - 1; i++) {
        //if(arr[i].frequency == -1 && i < size - 2) continue;
        k = i;
        for (j = i + 1; j < size; j++) {
            //if(arr[j].score == -1 && j < size - 1) continue;
            if (arr[j].frequency > arr[k].frequency) {
                k = j;
            }
        }
        if (k != i) {
            Swap(&arr[k], &arr[i]);
        }
    }
}

```

```
}
```

```
int FindAdminByCode(admin admins[], int n, const char *targetCode) {  
    for (int i = 0; i < n; i++) {  
        if (strcmp(admins[i].id, targetCode) == 0) {  
            printf("找到个人信息: \n");  
            printf("序号: %d\n", i+1);  
            printf("姓名: %s\n", admins[i].name);  
            printf("频率: %d\n", admins[i].frequency);  
            printf("编码: %s\n", admins[i].id);  
            return i;  
        }  
    }  
    printf("未找到匹配的编码: %s\n", targetCode);  
    return -1;  
}
```

```
int SearchCodeByName(admin admins[], int n, char *name) {  
    for (int i = 0; i < n; i++) {  
        if (strcmp(admins[i].name, name) == 0) {  
            printf("姓名: %s, 频率: %d, 编码: %s\n", admins[i].name,  
admins[i].frequency, admins[i].id);  
            return i;  
        }  
    }  
    printf("未找到姓名: %s\n", name);  
    return -1;  
}
```

```
void PrintCodeLengths(admin admins[], int n) {  
    printf("编码长度统计: \n");  
    for (int i = 0; i < n; i++) {  
        printf("%s: 编码长度 %d\n", admins[i].name, strlen(admins[i].id));  
    }  
}
```

```
void DisplayAllAdmins(admin admins[], int n) {  
    printf("所有个人信息: \n");  
    printf("序号\t姓名\t频率\t编码\n");  
    for (int i = 0; i < n; i++) {  
        printf("%d\t%s\t%d\t%s\n", i+1, admins[i].name, admins[i].frequency,  
admins[i].id);  
    }  
}
```

```
}
```

```
int main() {
    admin admins[28] = {{16, "赵", ""},{15, "钱", ""},{14, "孙", ""},{13, "朱", ""},{12, "李", ""},{11, "周", ""},{10, "吴", ""},{9, "王", ""},{25, "陈", ""},{23, "林", ""},{21, "徐", ""},{19, "郭", ""},{17, "何", ""},{16, "高", ""},{25, "桑", ""},{14, "马", ""},{12, "胡", ""},{11, "许", ""},{10, "邓", ""},{9, "萧", ""},{8, "程", ""},{7, "曹", ""},{6, "袁", ""},{5, "邓", ""},{4, "傅", ""},{3, "丁", ""},{2, "吕", ""},{1, "施", ""}};
    int n = 28;
    HuffmanTreeCode(admins, n);
    int choice;
    char searchName[MAX_NAME_LEN];
    do {
        printf("\n=== 哈夫曼编码管理系统 ===\n");
        printf("1. 显示所有编码\n");
        printf("2. 按频率排序显示\n");
        printf("3. 按姓名查找编码\n");
        printf("4. 显示编码长度\n");
        printf("5. 根据编码查找个人信息\n");
        printf("0. 退出\n");
        printf("请选择操作: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                DisplayAllAdmins(admins, n);
                break;
            case 2: {
                admin sortedAdmins[8];
                memcpy(sortedAdmins, admins, sizeof(admin) * n);
                SelectionSort(sortedAdmins, n);
                printf("按频率排序结果: \n");
                DisplayAllAdmins(sortedAdmins, n);
                break;}
            case 3:
                printf("请输入要查找的姓名: ");
                scanf("%s", searchName);
                SearchCodeByName(admins, n, searchName);
                break;
            case 4:
                PrintCodeLengths(admins, n);
                break;
            case 5: {
                printf("请输入要查找的哈夫曼编码: ");
```

```

        char searchCode[MAX_ID_LEN];
        scanf("%s", searchCode);
        FindAdminByCode(admins, n, searchCode);
        break;}
    case 0:
        printf("感谢使用\n");
        break;
    default:
        printf("无效选择, 请重新输入.\n");
    }
} while(choice != 0);
return 0;
}

```

四、上机结果及体会

1、 实际完成的情况说明（完成的功能，支持的数据类型等）

主要内容是哈夫曼编码的编写与解码，然后加入了生活中的元素“球员在网络应援出现的频率”联系到生活实际中来，可以通过哈夫曼编码来查找球员的信息，主要运用到了二叉树，但不是一般的二叉树，是静态的二叉树。

2、 程序的性能分析：

主要构建哈夫曼树的时间复杂度为： $O(n^2)$

原因：需要执行 $n-1$ 次合并操作，每次合并需遍历剩余节点寻找最小值

主要空间占用为编码表的占用，空间复杂度为： $O(n \log n)$

原因： n 个字符 $\times$ 平均编码长度 $O(\log n) = O(n \log n)$ 。

注记：平均编码长度为哈夫曼树的高度

3、 打印程序运行时的初值和运行结果，画出相应的图示：

```

=== 哈夫曼编码管理系统 ===
1. 显示所有编码
2. 按频率排序显示
3. 按姓名查找编码
4. 显示编码长度
5. 根据编码查找个人信息
0. 退出
请选择操作：1
所有个人信息：
序号    姓名    频率    编码
1       赵      16     111
2       钱      15     110
3       孙      14     101
4       朱      13     100
5       李      12     011
6       周      11     010
7       吴      10     001
8       王      9      000

```



```

=== 哈夫曼编码管理系统 ===
1. 显示所有编码
2. 按频率排序显示
3. 按姓名查找编码
4. 显示编码长度
5. 根据编码查找个人信息
0. 退出
请选择操作：2
按频率排序结果：
所有个人信息：
序号    姓名    频率    编码
1       赵      16     111
2       钱      15     110
3       孙      14     101
4       朱      13     100
5       李      12     011
6       周      11     010
7       吴      10     001
8       王       9     000

```

```

=== 哈夫曼编码管理系统 ===
1. 显示所有编码
2. 按频率排序显示
3. 按姓名查找编码
4. 显示编码长度
5. 根据编码查找个人信息
0. 退出
请选择操作：4
编码长度统计：
赵：编码长度 3
钱：编码长度 3
孙：编码长度 3
朱：编码长度 3
李：编码长度 3
周：编码长度 3
吴：编码长度 3
王：编码长度 3

```

```

=== 哈夫曼编码管理系统 ===
1. 显示所有编码
2. 按频率排序显示
3. 按姓名查找编码
4. 显示编码长度
5. 根据编码查找个人信息
0. 退出
请选择操作：3
请输入要查找的姓名：赵
姓名：赵，频率：16，编码：111

```

```

=== 哈夫曼编码管理系统 ===
1. 显示所有编码
2. 按频率排序显示
3. 按姓名查找编码
4. 显示编码长度
5. 根据编码查找个人信息
0. 退出
请选择操作：5
请输入要查找的哈夫曼编码：111
找到个人信息：
序号：1
姓名：赵
频率：16
编码：111

```

4、上机过程中出现的问题及其解决方案

问题：哈夫曼编码在当前用途比较窄

解决方法：查阅哈夫曼编码的优点，并且回忆比赛的过程，后来想到球员应援需要大量的重复信息交换，于是想到了哈夫曼编码可以在应援中传递信息，并且极大的降低了传输错误率。

问题：排序方式不同会降低时间与空间复杂度

解决方法：改用更加快捷简便的排序方式

5、程序中可以改进的地方说明

(1) 改用队列构建哈夫曼树，时间复杂度可进一步降低

(2) 使用动态编码表存储，可减少编码表空间占用

6、程序中可以扩充的功能及设计实现假想

(1) 进一步扩展的部分与应用假想

(2) 可以将此哈夫曼编码进一步加密，确保数据的安全性

(3) 可以实时监测球员受欢迎的频率，进行跳频编码，更加符合实际情况

7、收获及体会

这次上机，自己想开发的一个系统，让我更加深层次的体会到了一个系统开发的过程，首先，需要了解这个数据结构可以干一些什么事情？比如哈夫曼编码，哈夫曼树，它的路径权重和最小，也就是传输的效率会更高，在通信领域，编码也是一个很重要的方面，特别是二进制编码，而哈夫曼编码（已经学过的二叉树）就是二进制编码，完美符合了这一点。虽然上课听了关于哈夫曼树的构建过程，而且也会自己用纸笔构建哈夫曼树，但是真正到了实践还是有比较大的问题的，后来经过查阅资料与改造应用之后，哈夫曼编码才真正完成。不过，这个仅仅构建哈夫曼树并且编码似乎系统过于简单了一些，于是我又给他加了一些功能，比如：通过编码查找人物，通

过人物查找编码，这样一来就成为了一个编码与解码的过程，中间的交流只需要通过二进制交流就可以了，而且效率会比一般的编码要高上许多。

五、参考文献

SecureCode.【数据结构】【哈夫曼树】哈夫曼树、赫夫曼树（Huffman Tree）C 语言实现
<https://blog.csdn.net/crr411422/article/details/129932889>
ZhangJiqun&Hoper.哈夫曼树（赫夫曼树、最优树）详解
https://blog.csdn.net/qq_38998213/article/details/132315974